МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Практическое занятие № 3

Тема занятия: работа со структурами данных: Списки и словари
Вариант: 11

Выполнил студент 3 курса группы
ЭФБО-09-23 Калинин Н.В.

Проверил Громов Е.В.

Москва
2026 г.

Цель работы: формирование у студентов устойчивых навыков работы со структурами данных List и Dictionary в среде Puzzle RPA для построения логики обработки, анализа и структурирования табличной информации.

В рамках работы студент должен:

- понять принципы хранения данных в виде списков и словарей;
- освоить создание и инициализацию коллекций;
- научиться преобразовывать строки Excel в объекты (словарь);
- освоить работу со структурой List[Dictionary];
- реализовать перебор данных через циклы;
- выполнять агрегацию, фильтрацию и накопление результатов;
- корректно обращаться к элементам по индексу и по ключу;
- формировать итоговые расчёты на основе собранных данных.

Результатом является разработанный сценарий, в котором реализованы структуры данных для хранения и обработки информации, используется список словарей для представления табличных данных, выполнен перебор и фильтрация элементов через циклы, а также реализованы расчёты, агрегация и формирование итогового аналитического отчёта, что позволяет автоматизировать обработку проектов и построить логическую модель работы робота в Puzzle RPA.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

В современном мире автоматизация бизнес-процессов стала неотъемлемой частью работы компаний любого масштаба. Каждый день организации обрабатывают огромные объёмы информации: таблицы с проектами, отчёты по расходам, списки клиентов, данные из веб-сервисов и систем учета. Ручная обработка таких данных занимает много времени, создаёт риск ошибок и не позволяет быстро реагировать на изменения. Именно поэтому технологии RPA и инструменты, такие как Puzzle RPA, активно используют структуры данных, которые позволяют роботу эффективно хранить, обрабатывать и анализировать информацию. Две ключевые структуры данных, с которыми сталкивается любой разработчик RPA, – это списки (List) и словари (Dictionary). Они помогают не просто «собирать данные», а организовывать их в логическую и легко управляемую форму. Особенно мощным инструментом является список словарей (List[Dictionary]), который позволяет хранить сразу целую таблицу данных, где каждая строка представлена как словарь с ключами – атрибутами. С такой структурой данных можно легко фильтровать проекты по статусу, подсчитывать суммы бюджетов, формировать отчёты и строить сложную логику робота, которая будет работать с данными так, как это делал бы человек, но гораздо быстрее и без ошибок.

Таким образом, понимание и умение работать со списками и словарями – это основа аналитической работы в Puzzle RPA. Оно позволяет студентам переходить от простого использования блоков к созданию интеллектуальной и масштабируемой логики, способной обрабатывать данные любого объёма и сложности.

Списки (List) в RPA

Списки и их особенности в RPA

Список – это упорядоченная коллекция элементов, которая позволяет хранить и обрабатывать множество данных в одной переменной.

Основные характеристики списка:

- Упорядоченность: каждый элемент имеет свой индекс, начиная с 0.
- Гибкость: элементы могут быть разных типов – числа, строки, даты, логические значения, словари и даже другие списки.

- Динамичность: можно добавлять, удалять и изменять элементы в процессе работы робота.

Списки – это фундаментальная структура данных в RPA, потому что почти все реальные задачи связаны с обработкой множества элементов: проекты, сотрудники, файлы, транзакции, уведомления и т.д.

Примеры типичных кейсов:

- Обработка Excel: перебор строк таблицы с проектами, задачами или расходами.
- API и веб-сервисы: получение списка клиентов, заказов, уведомлений.
- Файловая система: обработка списка файлов в папке для загрузки или анализа.
- Накопление данных: сохранение результатов расчётов в цикле.
- Фильтрация и аналитика: создание списков по определённым условиям – активные проекты, просроченные задачи, крупные бюджеты.

Интересные особенности списков в RPA

1. Списки как накопители данных

В RPA списки часто используются не просто для хранения, а для аккумуляции информации в процессе выполнения робота. Таким образом, список становится фильтром и накопителем, который потом можно использовать для уведомлений, отчётов или аналитики.

2. Вложенные списки (*List* внутри *List*)

Списки могут содержать другие списки, что позволяет хранить сложные структуры данных. В RPA это полезно, когда нужно работать с иерархическими данными: проект → задачи → подзадачи.

3. Динамические списки

Списки в RPA не имеют фиксированной длины – их можно расширять во время работы робота. Это позволяет роботу адаптироваться к изменениям данных, не ломая логику сценария.

4. Списки как «фильтры» и «условные коллекции»

Списки можно использовать для быстрой фильтрации и формирования условных коллекций. В RPA это удобно для:

- формирования уведомлений для руководства,
- подготовки списка проектов для отдельного сценария,
- автоматической сегментации данных.

5. Списки и сортировка

Списки можно сортировать, что помогает строить отчёты или делать аналитические выборки. В RPA это часто применяется для:

- построения сводных таблиц;
- определения, какие проекты требуют внимания в первую очередь;

- упрощения логики уведомлений.

6. Списки для многократного использования

Списки позволяют не дублировать данные и использовать один набор данных в нескольких сценариях. Все операции работают с одним источником данных, что повышает надёжность и упрощает поддержку робота.

7. «Фишки» списков для оптимизации. Списки можно фильтровать и сортировать в одну строку – сокращает код и делает сценарий аккуратным.

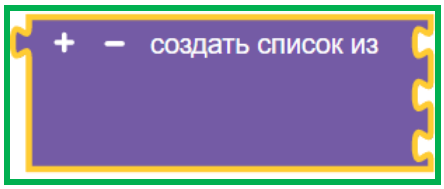
Можно использовать списки для логов и ошибок, аккумулируя все сообщения в один список и выводя их после завершения процесса.

Организация списков в Puzzle RPA

В Puzzle RPA работа со списками организована через блоки Стандартные → Массивы, которые позволяют создавать, изменять, анализировать и обрабатывать коллекции данных без необходимости писать сложный код.

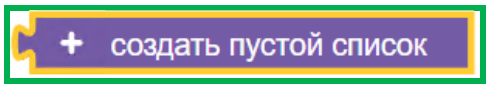
Ниже представлены все основные блоки с пояснениями.

1. Создать список из…



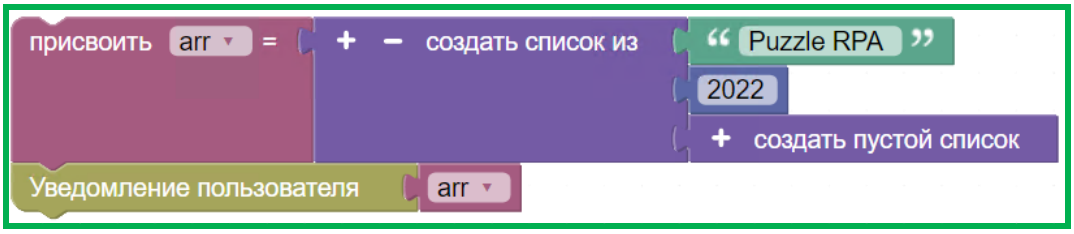
Назначение: создать новый список, указав элементы вручную или через переменные.

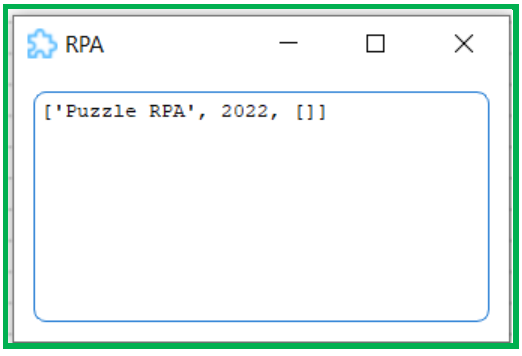
При удалении всех пазов через знак “-” можно создать пустой список.



Пример использования:

В данном примере блок создает список из строки, числа и пустого списка и присваивает переменной `arr`.





Применение в Puzzle RPA: удобно при инициализации данных перед циклом или перед фильтрацией.

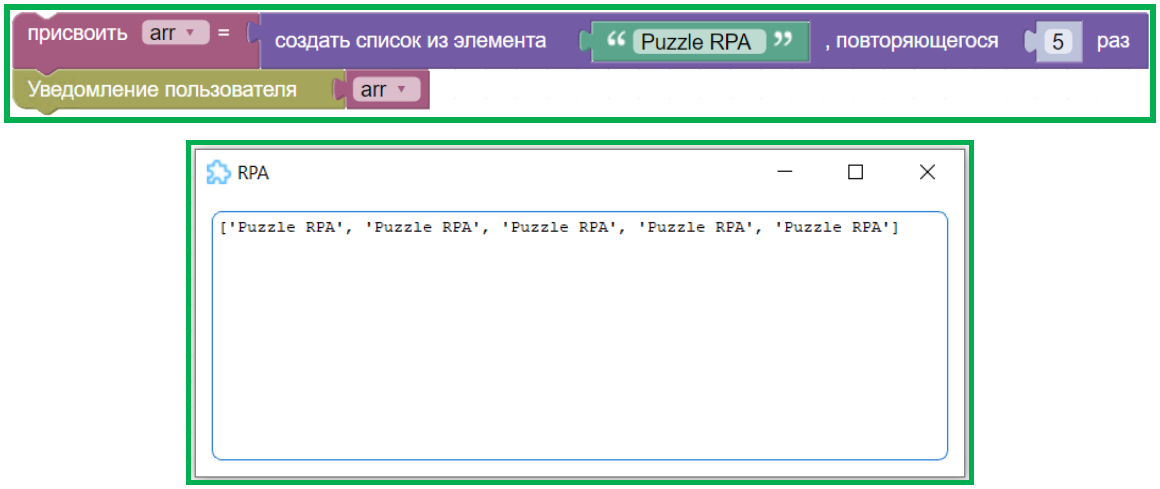
2. Создать список из повторяющегося элемента

Назначение: создать список, где один элемент повторяется несколько раз. Полезно для создания шаблонов или резервных списков.



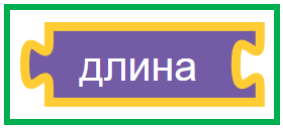
Пример использования:

В данном примере блок создает список из повторяющейся строки и присваивает переменной `arr`.



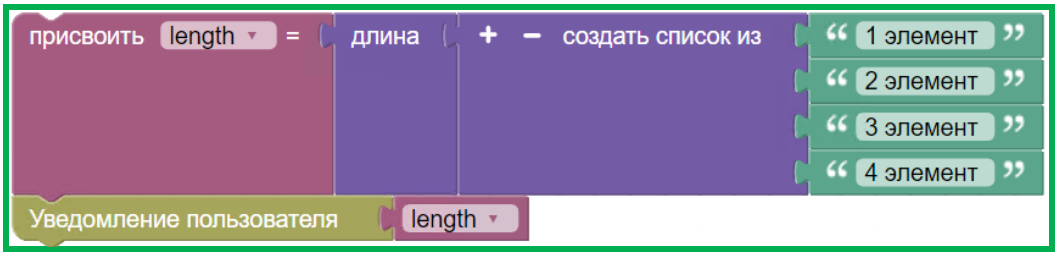
3. Длина списка

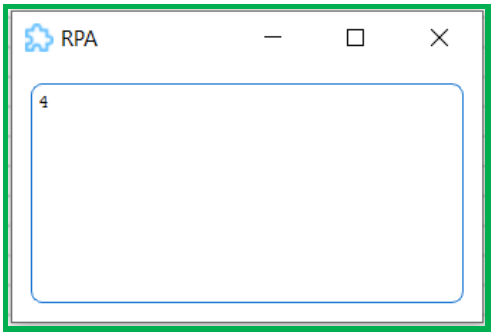
Назначение: возвращает количество элементов в списке. Используется для контроля количества элементов перед циклом или для проверки, пуст ли список.



Пример использования:

В данном примере блок присваивает переменной `length` длину массива.





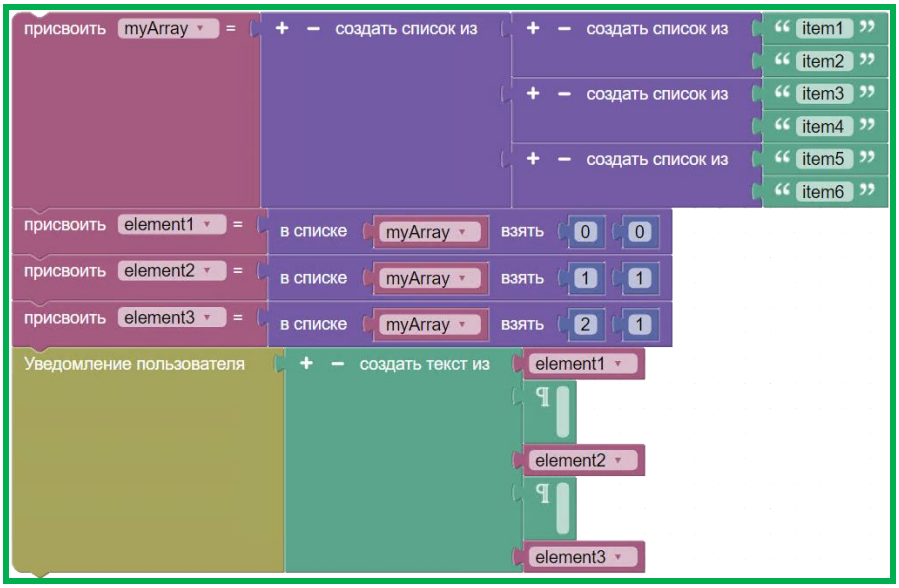
4. Элемент двумерного массива

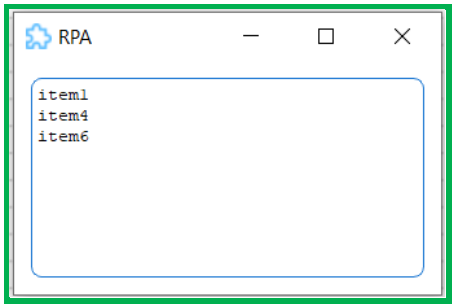
Назначение: позволяет получить или изменить элемент в двумерном списке (список списков). Полезно, когда каждая строка содержит несколько атрибутов проекта (например, название и статус).



Пример использования:

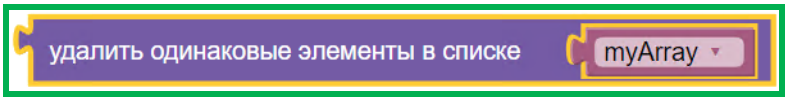
В данном примере блок возвращает 3 элемента двумерного массива `myArray`.





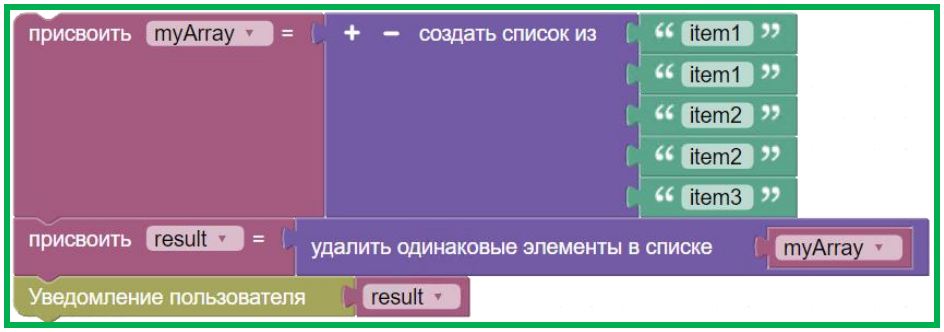
5. Удалить дубликаты

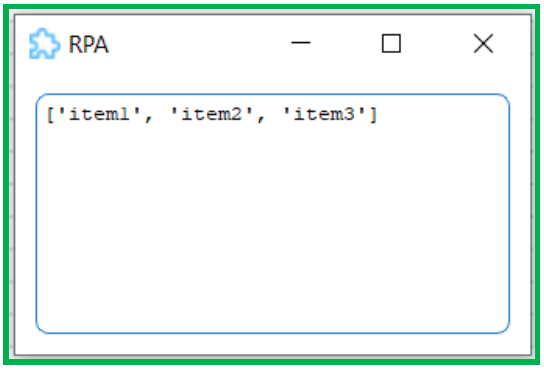
Назначение: очищает список от повторяющихся элементов. Используется для анализа уникальных значений, например, уникальных статусов или клиентов.



Пример использования:

В данном примере блок возвращает список уникальных элементов





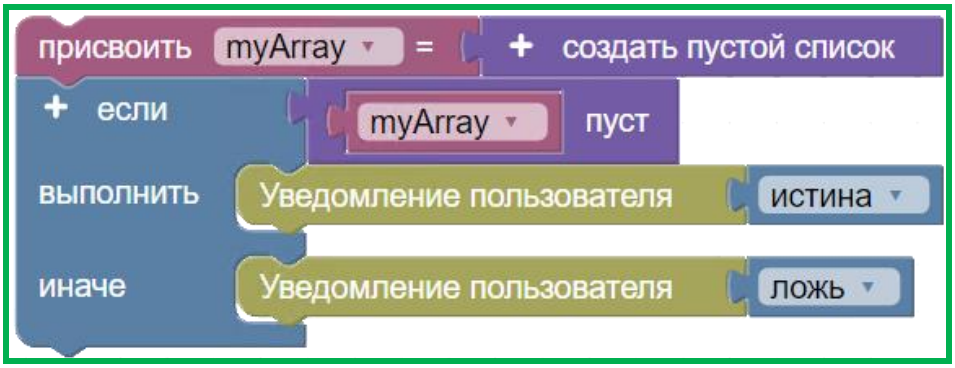
6. Пуст ли список

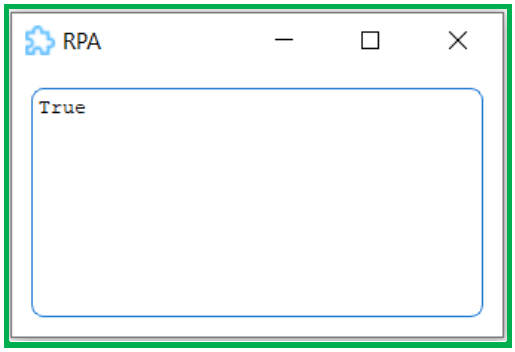
Назначение: проверяет, содержит ли список элементы или нет. Возвращает True/False. Часто используется перед циклом, чтобы не пытаться перебрать пустой список.



Пример использования:

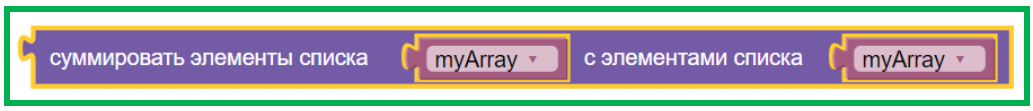
В данном примере блок возвращает значение true, потому что список myArray пустой.





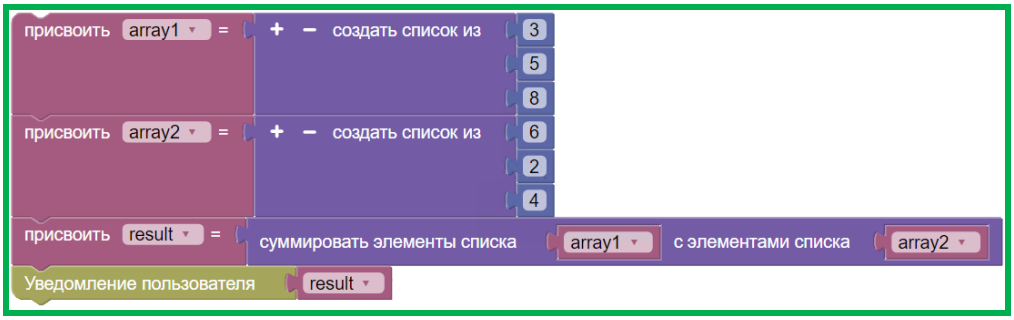
7. Суммировать элементы списка

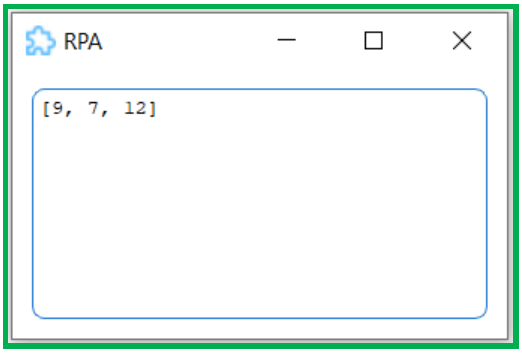
Назначение: вычисляет сумму всех числовых элементов списка. Полезно для подсчёта суммарного бюджета проектов, расходов или транзакций.



Пример использования:

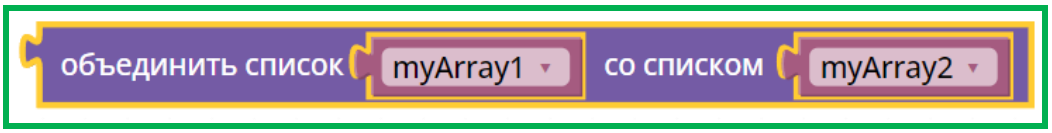
В данном примере блок суммирует элементы двух массивов по соответствующим индексам.





8. Объединить списки

Назначение: соединяет два или более списков в один. Используется для объединения результатов из разных источников или после фильтрации.



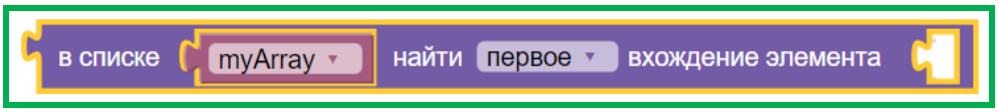
Пример использования:

В данном примере блок объединяет два списка и выводит результирующий список в уведомлении.



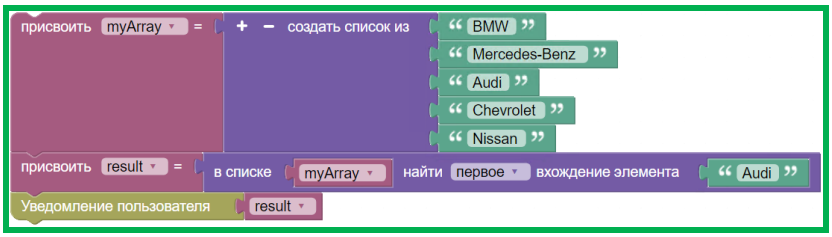
9. Найти номер элемента в списке.

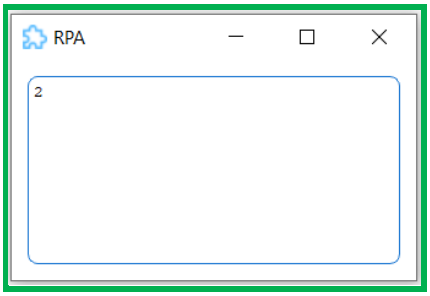
Назначение: возвращает индекс элемента в списке. Возвращает -1 если элемент не найден. Удобно для прямого доступа к элементу или замены значения по индексу.



Пример использования:

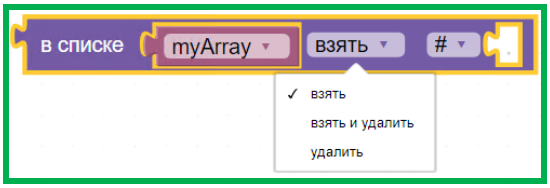
В данном примере блок объединяет два списка и выводит результирующий список в уведомлении.

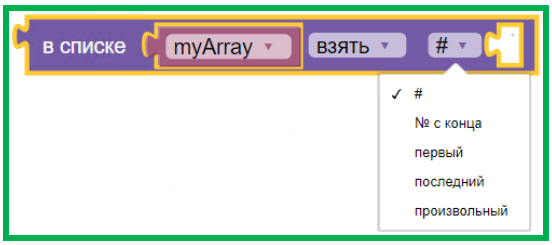




10. Взять/удалить элемент списка

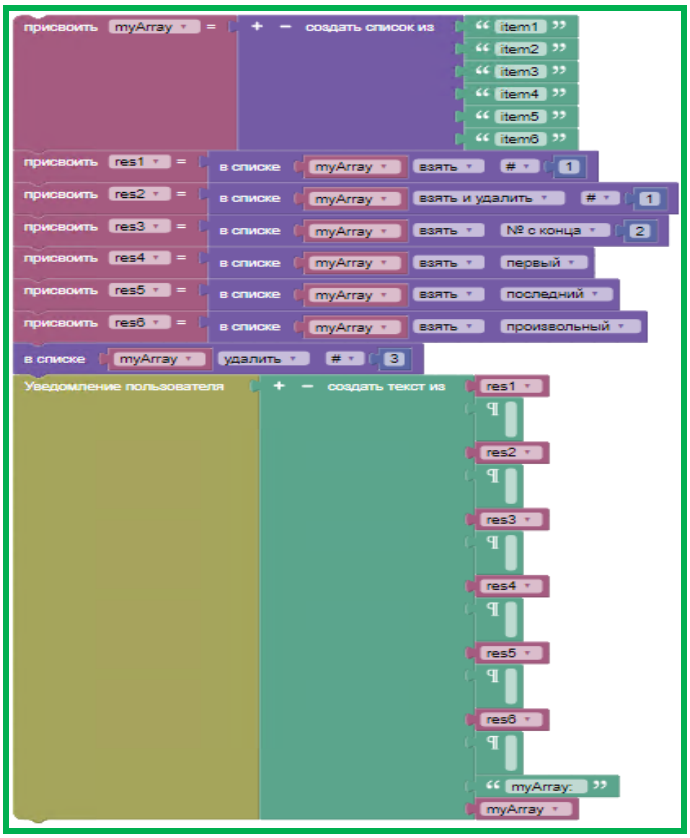
Назначение: позволяет взять элемент по индексу и при необходимости удалить его из списка. Полезно для обработки очереди задач, когда нужно «взять» элемент и исключить его из списка.

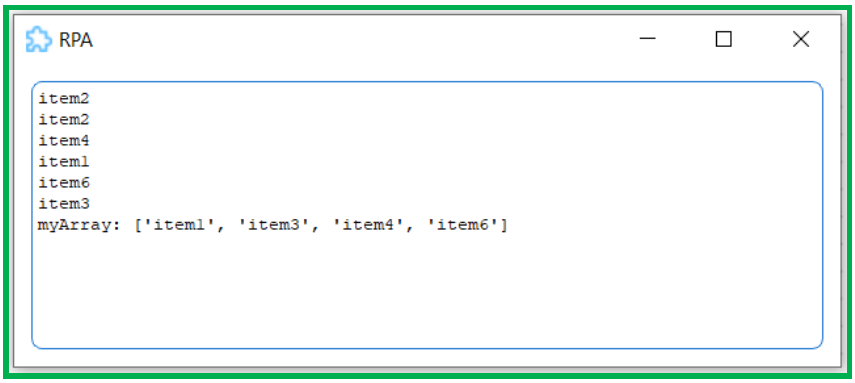




Пример использования:

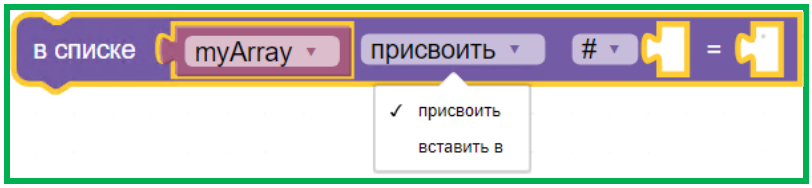
В данном примере выводится уведомление с результатом работы блока, в зависимости от параметра.





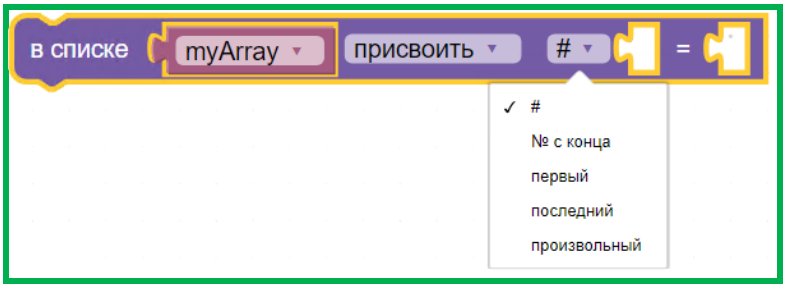
11. Присвоить значение элементу списка

Назначение: заменить значение конкретного элемента по индексу. Используется для исправления данных или обновления статусов проектов.



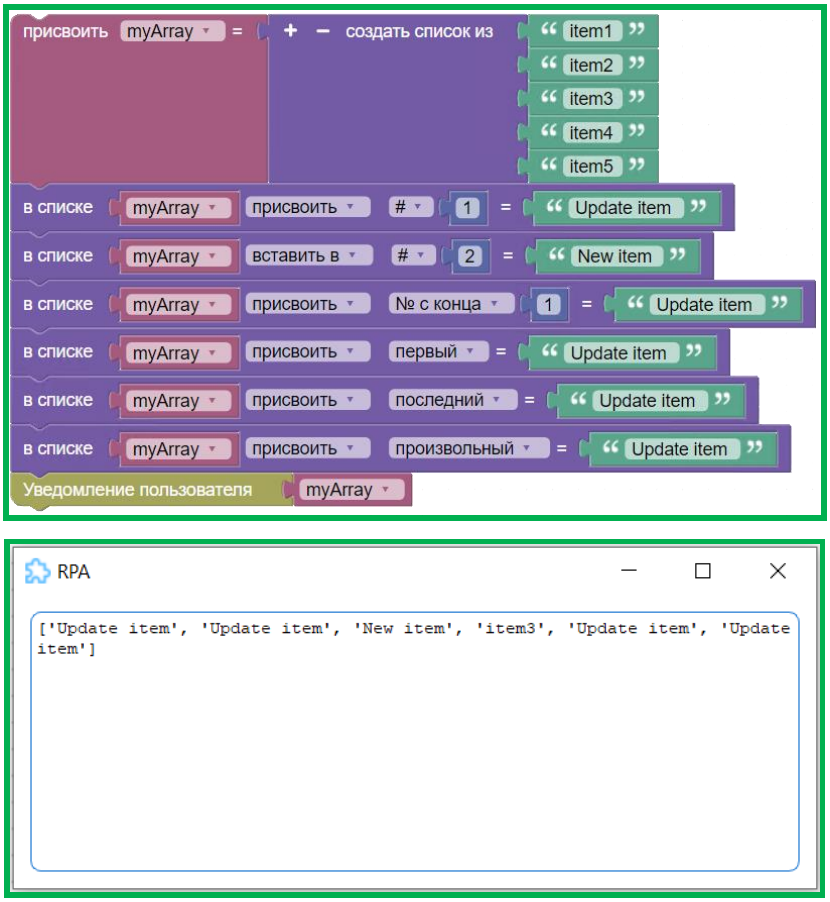
В меню представлены следующие опции:

- присвоить – заменить элемент в списке по индексу;
- вставить в – добавить элемент в список по индексу.



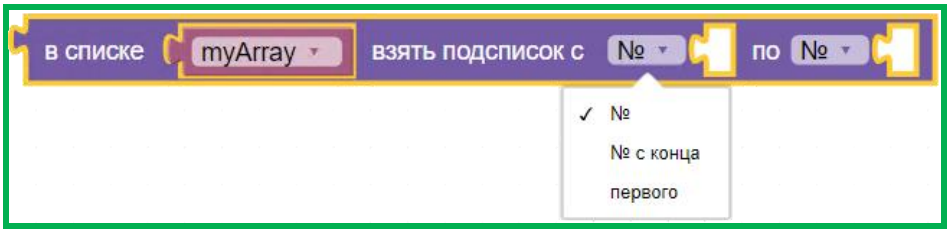
Пример использования:

В данном примере выводится уведомление с результатом работы блока, в зависимости от параметров.



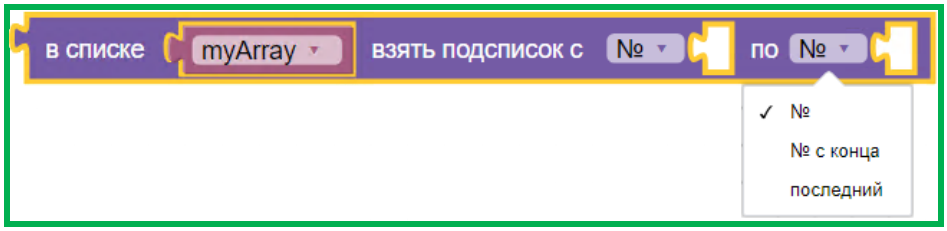
12. Взять подсписок списка

Назначение: получить часть списка (срез) по индексам. Удобно для обработки только части данных, например, проектов, которые попадают в текущий квартал.



В меню представлены следующие опции:

- № – возвращается элемент списка по индексу;
- № с конца – возвращается элемент списка по индексу с конца;
- первого – возвращает первый элемент списка.

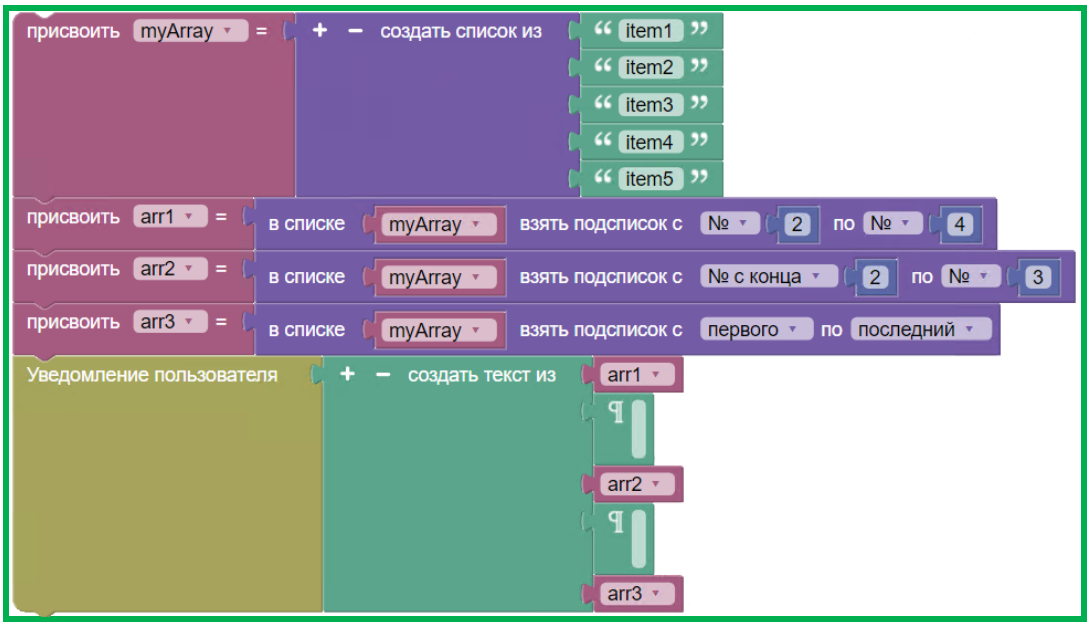


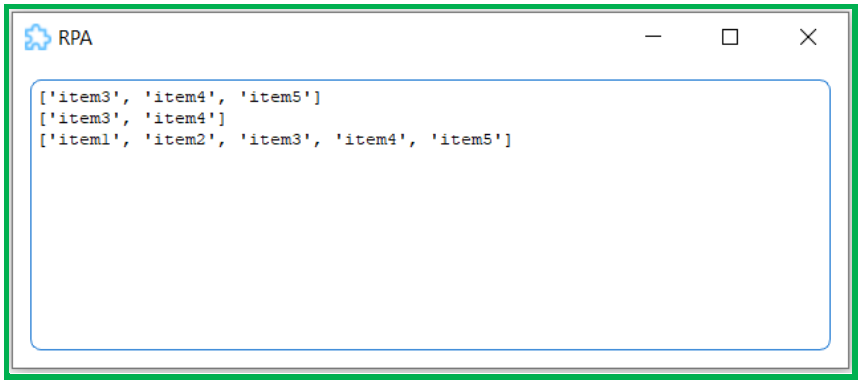
В меню представлены следующие опции:

- № – возвращается элемент списка по индексу;
- № с конца – возвращается элемент списка по индексу с конца;
- последний – возвращает последний элемент списка.

Пример использования:

В данном примере выводится уведомление с результатом работы блока, в зависимости от параметров.





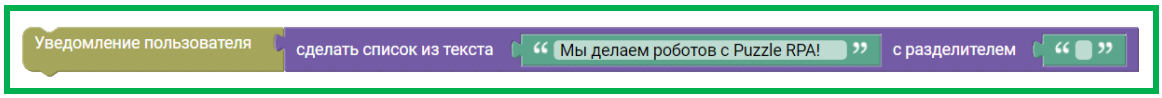
13. Сделать список из текста

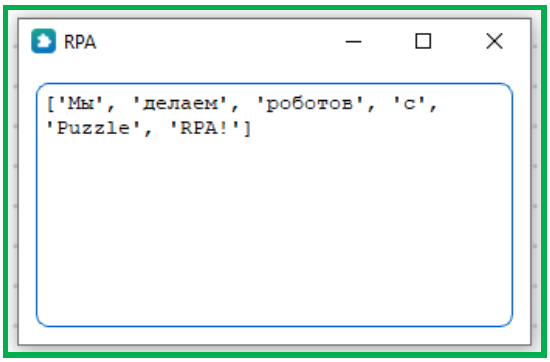
Назначение: разбивает текст на элементы списка по разделителю. Часто используется при работе с CSV или строковыми значениями из Excel или API.



Пример использования:

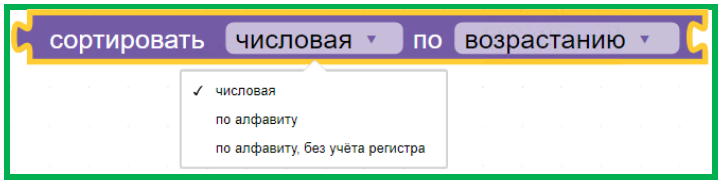
Блок разбивает текст в список текстов с указанным разделителем. В данном примере выводится уведомление с результатом работы блока.





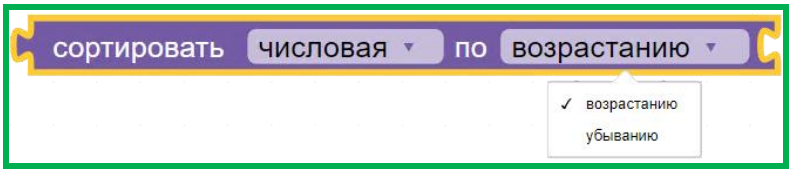
14. Сортировать список

Назначение: упорядочивает элементы списка по возрастанию или убыванию. В RPA применяется для подготовки отчётов или выбора проектов с максимальным/минимальным значением.



В меню представлены следующие опции:

- числовая – возвращается отсортированный список по числовому значению;
- по алфавиту – возвращается отсортированный список по алфавиту;
- по алфавиту, без учёта регистра – возвращается отсортированный список по алфавиту, без учета регистра.

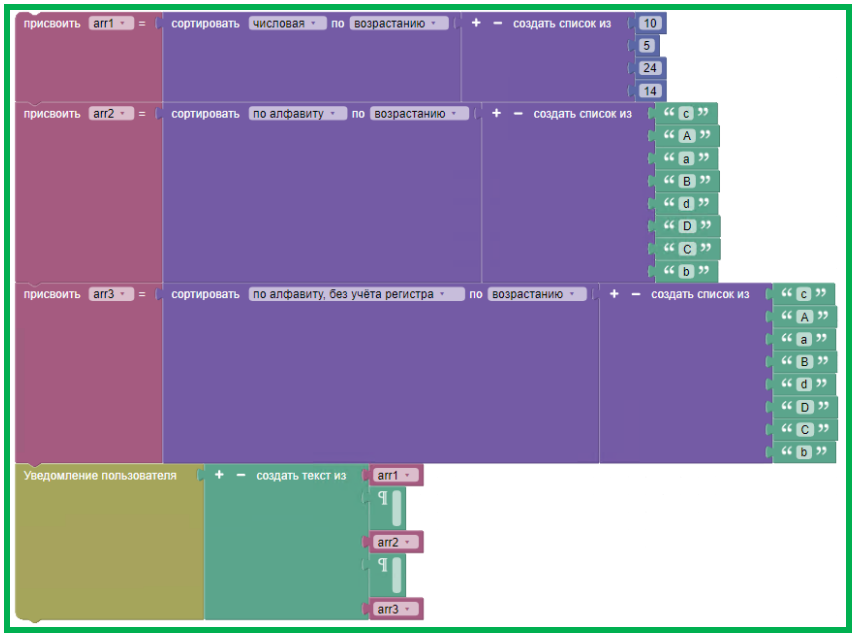


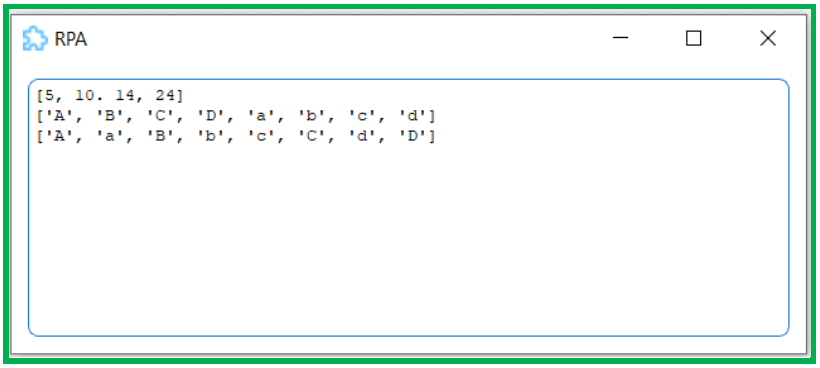
В меню представлены следующие опции:

- возрастанию – возвращается отсортированный список по возрастанию;
- убыванию – возвращается отсортированный список по убыванию.

Пример использования:

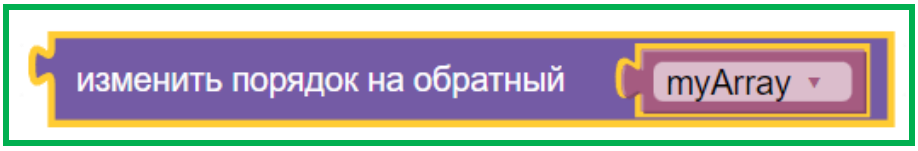
В данном примере выводится уведомление с результатом работы блока, в зависимости от параметров.





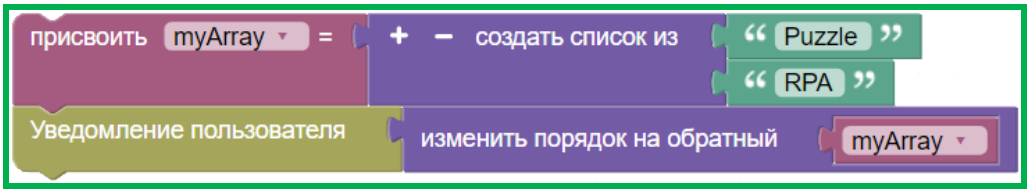
15. Изменить порядок

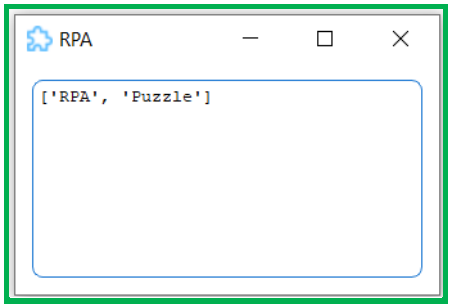
Назначение: переворачивает список, меняя порядок элементов на обратный. Используется для обработки очередей, когда нужно начать с последнего элемента, или для удобного отображения данных в отчёте.



Пример использования:

В данном примере блок меняет значения списка местами.





Все эти блоки позволяют гибко и безопасно работать со списками в Puzzle RPA, не прибегая к сложному коду.

Студенты, освоившие эти блоки, могут:

- создавать и инициализировать списки;
- фильтровать и обрабатывать элементы;
- изменять, объединять и сортировать данные;
- готовить списки для аналитики и отчётов;
- реализовывать сложные сценарии обработки данных.

Практический совет: в реальных задачах списки чаще всего используются вместе со словарями, чтобы строить мощные структуры *List[Dictionary]* для работы с таблицами и API.

Словари (Dictionary) в RPA

Словари и их назначение

В RPA словари используются для хранения данных в формате:

```
Ключ → Значение
```

Если список – это просто последовательность элементов, то словарь – это структурированный объект с атрибутами.

Именно словари позволяют роботу работать с данными так, как человек работает с карточкой объекта:

```
Проект → Название, Статус, Бюджет, Дата окончания
Сотрудник → Имя, Должность, Отдел
Транзакция → Номер, Сумма, Дата
```

В реальной автоматизации почти все данные – это объекты с параметрами.

Словарь – это:

- модель объекта
- структурированная единица данных
- логическая сущность

Когда мы начинаем мыслить объектами, а не строками таблицы, мы переходим на следующий уровень разработки.

Что такое словарь в контексте RPA

- Словарь – это структура:

```
ключ → значение
```

Пример объекта «Проект»:

```
{
   "Name": "CRM System",
   "Status": "Active",
   "Budget": 150000,
```

```
    "EndDate": "2026-03-01"
}
```

- Ключ – это имя поля.
- Значение – это данные.

Почему это важно? Потому что теперь робот может обращаться к данным логически:

```
project["Status"]
project["Budget"]
```

А не через непонятные индексы:

```
row[2]
row[3]
```

Где словари используются в RPA?

1. Работа с Excel . Каждая строка таблицы логически превращается в словарь.

2. Работа с API. Большинство API возвращают данные в формате JSON – а JSON по сути и есть словарь.

3. Формирование отчётов. Словарь удобно использовать как объект отчёта перед отправкой.

4. Расчёт аналитики. Словарь можно дополнять новыми вычисленными полями.

Интересные аспекты словарей в RPA:

1. Словарь как «живая модель данных». В RPA словарь – это не просто контейнер. Это объект, который развивается по мере выполнения процесса.

Представим сценарий:

- Прочитали Excel → создали словарь проекта
- Посчитали длительность проекта → добавили новое поле
- Рассчитали отклонение бюджета → добавили ещё поле
- Определили риск → добавили уровень риска

В начале:

```
{
  "Name": "CRM",
  "Budget": 100000
}
В конце процесса:
{
  "Name": "CRM",
  "Budget": 100000,
  "DurationDays": 180,
```

```
  "DeviationPercent": 12,
  "RiskLevel": "Medium"
}
```

Это называется обогащение объекта.

Именно так строится интеллектуальная автоматизация.

2. Словарь как инструмент группировки данных. Очень мощная техника – использовать словарь для группировки. Например, нужно сгруппировать проекты по статусу.

Исходные данные:

```
Active, Completed, Active, On Hold
```

Можно создать словарь:

```
{
  "Active": [проект1, проект3],
  "Completed": [проект2],
  "On Hold": [проект4]
}
```

Это называется группировка через словарь.

В RPA это применяется для:

- аналитических отчётов
- построения сводной информации
- расчёта показателей по категориям

3. Словарь как счётчик (Counter-подход). Словарь можно использовать для подсчёта значений.

4. Динамические ключи. В словаре ключи могут создаваться динамически во время выполнения. Это особенно полезно, когда:

- заранее неизвестны категории,
- данные приходят из API,
- структура может меняться.

5. Словарь как кэш (ускорение процессов). Иногда в RPA нужно часто проверять наличие значения. Словарь позволяет делать это очень быстро. Это важно при обработке больших объёмов данных.

6. Вложенные словари – моделирование сложных систем. В реальном бизнесе данные сложные. Проект может содержать:

- Финансы
- Команду
- Сроки

7. Словарь как конфигурация процесса. Очень интересный приём – использовать словарь для хранения настроек. Чтобы изменить логику – не нужно менять блок, достаточно изменить конфигурацию.Это профессиональный подход к разработке.

8. Словарь и JSON – мост к API. Большинство API работают с JSON. А JSON – это по сути словарь. Поэтому если студент освоил словари, он уже:

- готов к интеграциям,
- понимает структуру API,
- может формировать запросы и читать ответы.

Пример ответа API:

```
{
  "id": 101,
  "status": "approved",
  "amount": 5000
}
```

Это обычный словарь.

9. Защита от ошибок через словарь. В RPA часто встречаются:

- пустые значения
- отсутствующие поля
- изменяющиеся структуры

Словарь позволяет строить защитную логику: Если ключа нет – берём 0. Процесс не падает. Это называется устойчивость сценария.

10. Архитектурное мышление. Когда студент работает со словарями, он начинает:

```
мыслить объектами,
структурировать данные,
проектировать модели,
строить масштабируемые процессы.
```

Это переход от: «Я перебираю строки Excel» к «Я работаю с моделью проекта»

Главное понимание: словарь в RPA – это:

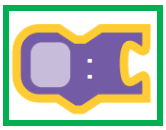
- модель объекта
- инструмент аналитики
- способ группировки
- счётчик
- кэш
- конфигурация
- основа API-интеграции

- механизм устойчивости процесса

Основные блоки работы со словарями

Рассмотрим основные блоки работы со словарём в Puzzle RPA.

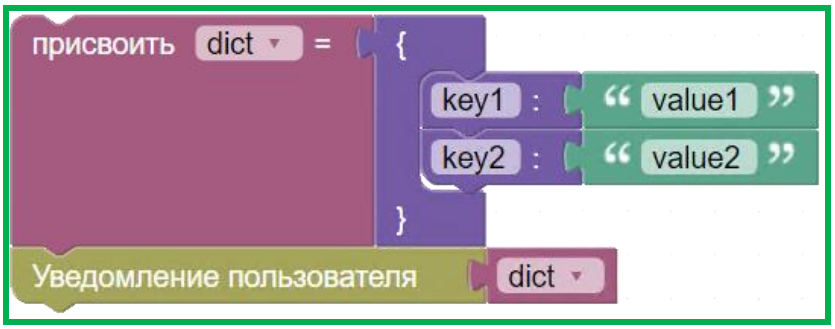
1. Ключ–значение (основа словаря)

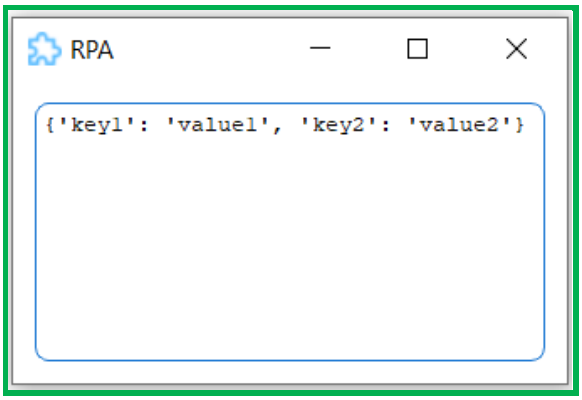


Словарь строится на принципе соответствия: уникальный ключ → связанное значение. Ключ – это имя свойства. Значение – данные, связанные с этим свойством. Один ключ не может повторяться. Значения могут быть разных типов.

Пример использования:

В данном примере выводится уведомление с результатом работы блока.





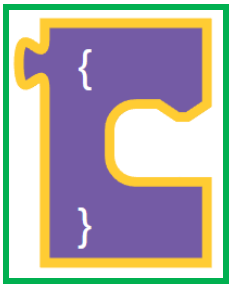
2. Создать словарь

Создаётся переменная типа «Словарь». Формируется контейнер для хранения структурированных данных.

Когда используется:

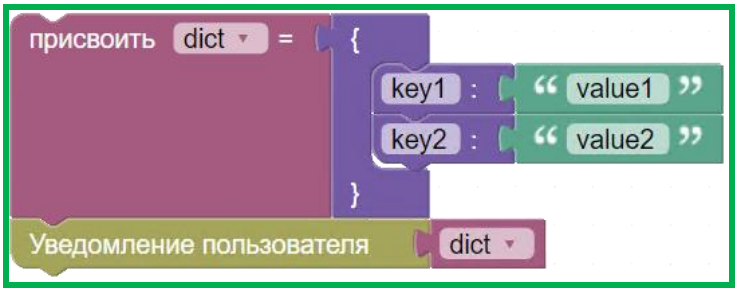
- Перед заполнением данных
- Перед циклом обработки
- Перед накоплением информации
- При создании объекта

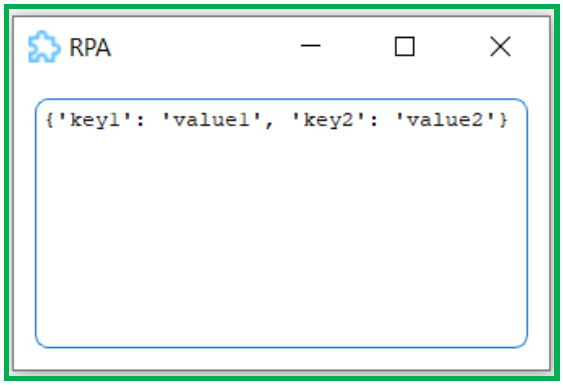
Создание словаря – это создание модели будущего объекта.



Пример использования:

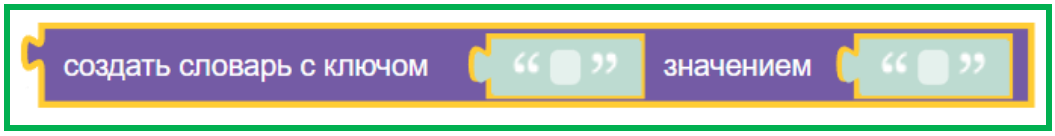
В данном примере создается словарь.





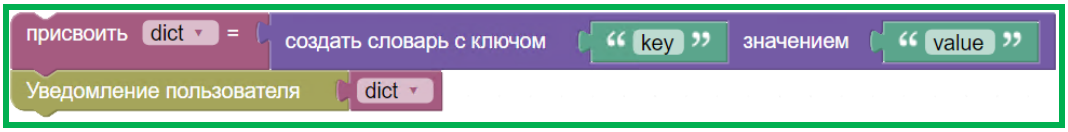
3. Создать словарь с ключом-значением

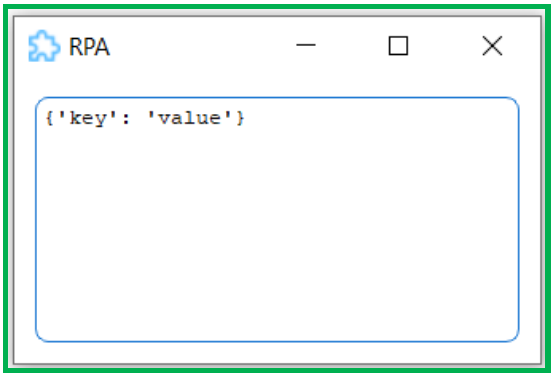
Создаётся словарь и сразу задаётся его структура – определяются ключи и их начальные значения. Используется: когда структура заранее известна; при создании конфигурации процесса; при формировании шаблона объекта. Это уже не просто контейнер, а заранее определённая модель данных.



Пример использования:

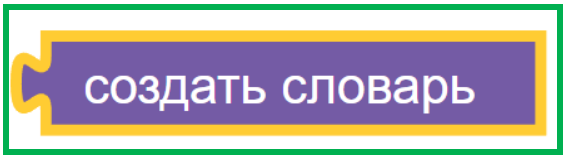
В данном примере выводится уведомление с результатом работы блока.





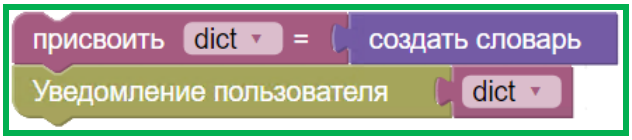
4. Создать пустой словарь

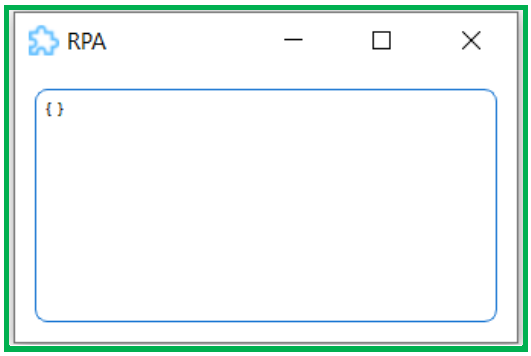
Создаётся словарь без ключей и значений. Используется когда структура будет формироваться динамически: при группировке данных; при создании счётчиков; при накоплении информации в цикле. Пустой словарь – это гибкая структура, которая будет расширяться во время выполнения процесса.



Пример использования:

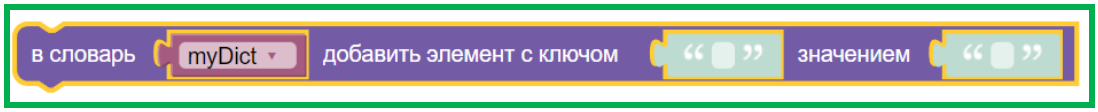
В данном примере выводится уведомление с результатом работы блока.





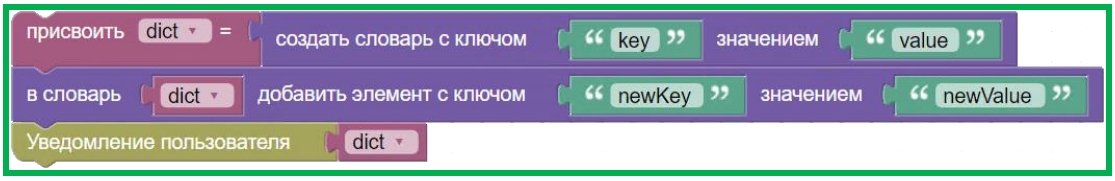
5. Добавить элемент в словарь

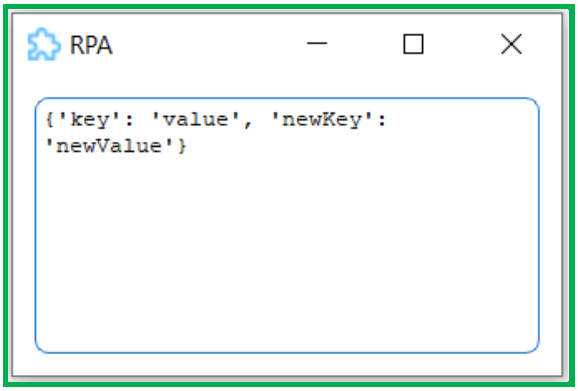
В словарь добавляется новая пара ключ–значение либо обновляется существующая. Если ключа не было – он создаётся. Если ключ был – значение перезаписывается. Используется при обогащении объекта новыми данными; при расчётах; при накоплении аналитики; при формировании результатов. Словарь динамический – его структура может изменяться в любой момент выполнения.



Пример использования:

В данном примере выводится уведомление с результатом работы блока.





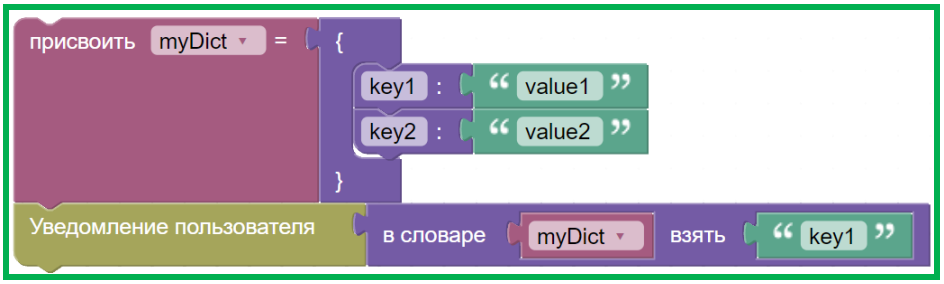
6. В словаре взять значение

Получение значения по указанному ключу. Если ключ отсутствует – возможна ошибка. Поэтому часто используется предварительная проверка. Используется в расчётах; условиях; при формировании отчётов; при логике принятия решений. Получение значения – это доступ к свойству объекта.



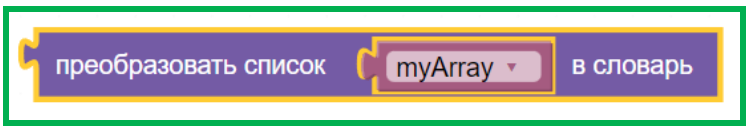
Пример использования:

В данном примере создается словарь, производится поиск значения в словаре по ключу “key1”. Найденное значение выводится на экран.



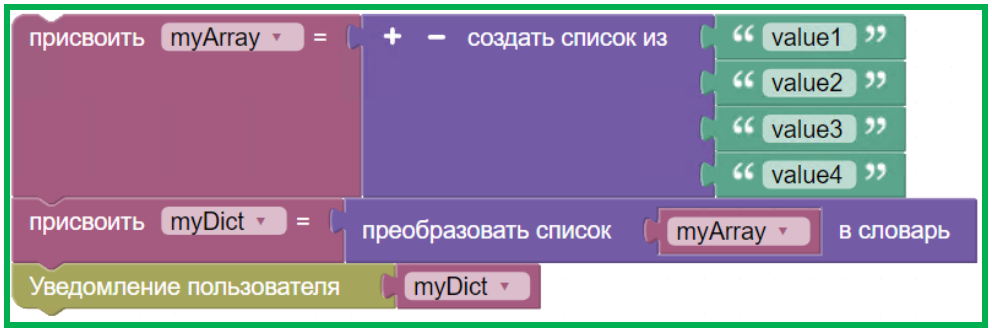
7. Преобразовать список в словарь

Структура типа «список» преобразуется в структуру «словарь». Используется при работе с данными, полученными из внешних источников; при интеграциях; при обработке структурированных пар данных. Это изменение формы хранения данных без изменения их сути.



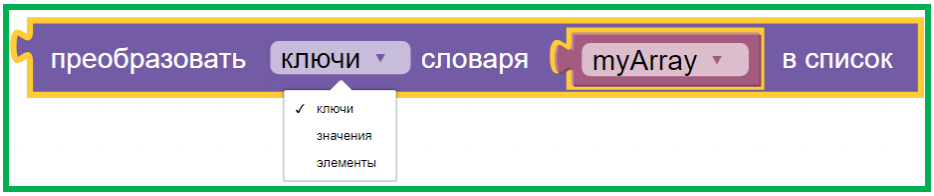
Пример использования:

В данном примере выводится уведомление с результатом работы блока.



8. Преобразовать словарь в список

Словарь переводится в список ключей, список значений или список пар. Когда используется: Для перебора данных; Для экспорта; Для логирования; Для передачи в другие блоки, работающие со списками. Иногда удобнее работать не со структурой «ключ–значение», а с линейной последовательностью данных.

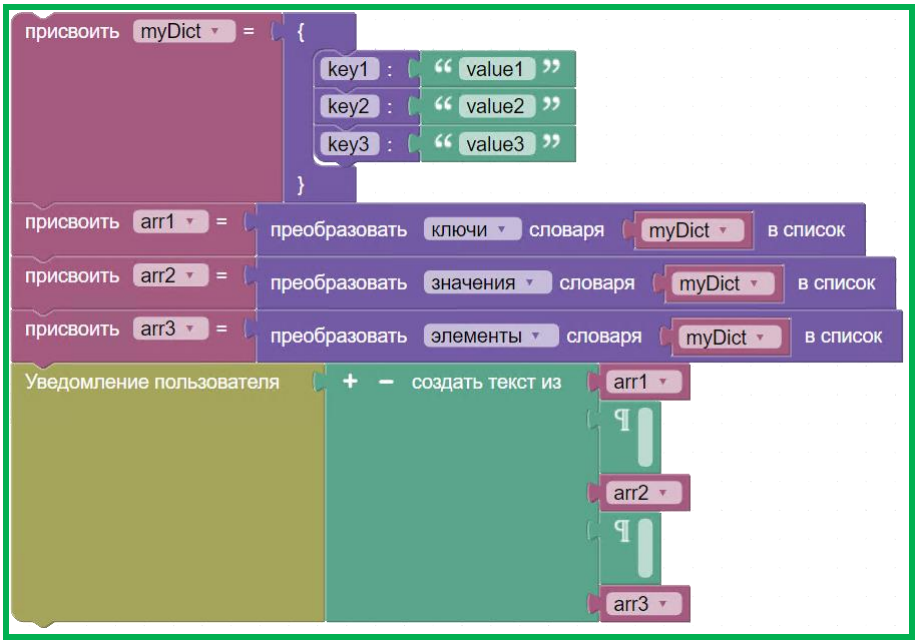


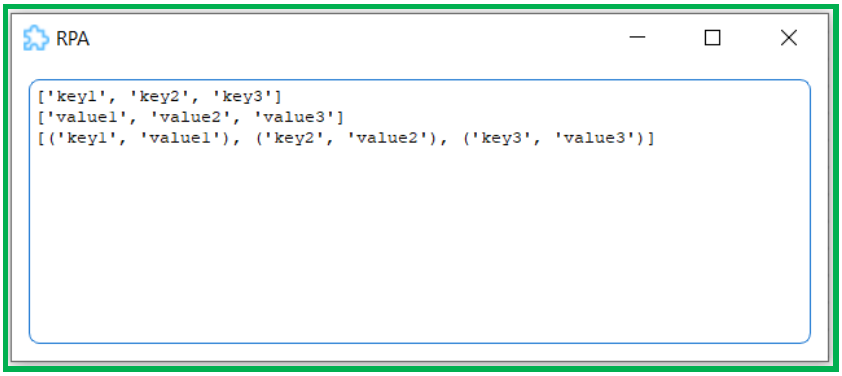
В меню представлены следующие опции:

- ключи–возвращается список с ключами словаря;
- значения–возвращается список со значениями словаря;
- элементы–возвращается список с элементами словаря.

Пример использования:

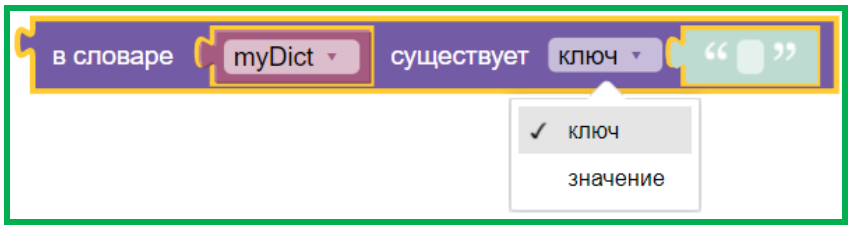
В данном примере выводится уведомление с результатом работы блока.





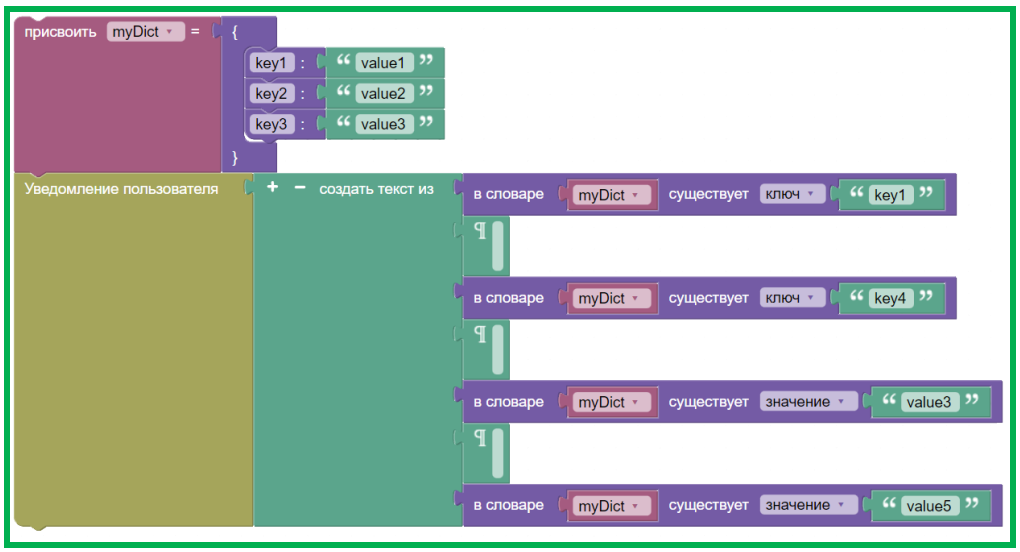
9. Существует ли ключ в словаре

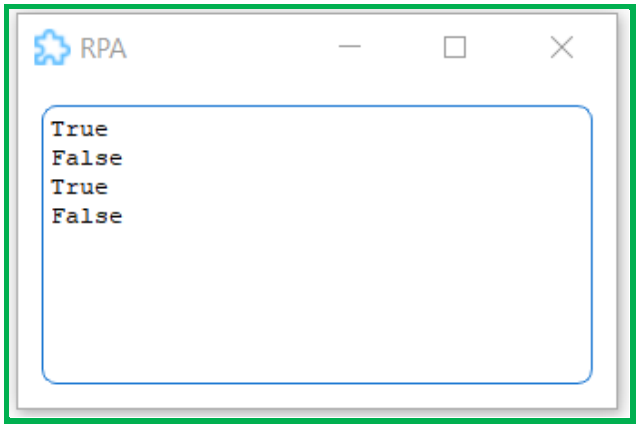
Проверяется наличие определённого ключа в словаре. Позволяет избежать ошибок. Делает процесс устойчивым. Позволяет строить гибкую логику. Используется: Перед получением значения; При динамической структуре данных; При работе с API; При нестабильных источниках данных. Это элемент защитной архитектуры процесса.



Пример использования:

В данном примере выводится уведомление с результатом работы блока.





Общая логика работы со словарём в Puzzle RPA:

Все блоки делятся на 4 категории:

1. Создание структуры

- Создать словарь
- Создать пустой словарь
- Создать словарь с ключами

2. Изменение структуры

- Добавить элемент
- Обновить значение

3. Чтение данных

- Взять значение
- Проверить существование ключа

4. Преобразование структуры

- Список → словарь
- Словарь → список

Работа со словарями в Puzzle RPA – это не просто технический навык, а переход к структурному и архитектурному мышлению.

Через словарь робот перестаёт быть «исполнителем последовательных действий» и становится системой.

Важно понимать: если список отвечает на вопрос «что у нас есть», то словарь отвечает на вопрос «какими свойствами это обладает». Именно поэтому словари лежат в основе аналитических процессов.

Комбинирование словарей и списков в RPA

До этого мы рассматривали:

- список – как последовательность элементов
- словарь – как модель объекта

Но в реальных RPA-проектах они почти никогда не используются отдельно. Настоящая архитектура строится на связке: Список словарей. Именно эта структура лежит в основе:

- обработки Excel
- анализа портфеля проектов
- CRM-данных
- финансовых отчётов
- работы с API

Это фундамент профессиональной автоматизации.

Список словарей – это структура вида: Список → хранит множество объектов; Каждый элемент списка → словарь; Каждый словарь → описание одного объекта

Логика:

```
{ [объект1], [объект2], [объект3] }
```

Где каждый объект – это словарь с набором свойств. Главная идея: список хранит структуру, словарь хранит свойства. Список отвечает за масштаб. Словарь отвечает за глубину.

Аналитика через словарь внутри списка позволяет:

- считать показатели
- группировать объекты
- строить сводную статистику

Например: есть список проектов; проходим по списку; используем дополнительный словарь как счётчик.

Получается:

- один список – данные
- один словарь – аналитика

Это уже многослойная архитектура.

Профессиональный уровень – это вложенность. Возможны варианты:

- Вариант 1. Список словарей
- Вариант 2. Словарь со списками
- Вариант 3. Словарь со списками словарей

Это уже моделирование сложных бизнес-систем. Комбинирование списков и словарей позволяет:

- добавлять новые поля без изменения структуры
- масштабировать процесс
- усложнять аналитику
- добавлять уровни вложенности

Без изменения базовой архитектуры.

Типовая логика реального RPA-проекта:

- Источник данных (Excel, API, БД)

- Формирование списка словарей
- Обогащение объектов
- Аналитика через дополнительные словари
- Формирование отчёта
- Уведомление пользователя
- Практически любой аналитический процесс строится по этой схеме.

Итоги: важные моменты

1. Список словарей – базовая архитектура RPA. Это фундамент обработки табличных данных.

2. Словарь – это модель объекта. Без словаря невозможно построить устойчивую логику.

3. Объекты можно обогащать в процессе. Словарь динамический – это ключ к интеллектуальной автоматизации.

4. Дополнительные словари создают аналитику. Счётчики, группировки, агрегаторы – всё это словари.

5. Комбинирование = переход на системный уровень. Студент начинает мыслить не блоками, а архитектурой данных.

Комбинирование списков и словарей в Puzzle RPA – это:

- основа построения сложных процессов
- фундамент аналитических задач
- механизм масштабируемости
- способ моделирования бизнес-структур
- архитектурный уровень автоматизации

Мы последовательно разобрали три фундаментальных блока:

- Словарь – как модель объекта
- Список – как контейнер множества объектов
- Комбинирование списка и словаря – как архитектурную основу RPA-процесса

Теперь важно зафиксировать главное понимание.

1. Словарь – это модель бизнес-сущности

Словарь позволяет:

- описывать объект через его свойства
- динамически расширять структуру
- хранить состояние процесса
- обеспечивать устойчивость логики

Это не просто структура данных – это цифровое представление реального объекта внутри автоматизации.

2. Список – это масштаб

Список отвечает за:

- хранение множества объектов
- последовательную обработку данных
- перебор элементов в цикле
- организацию потока информации

Если словарь – глубина, то список – масштаб.

3. Комбинирование – это архитектура. Связка «список словарей» – базовая модель практически любого аналитического процесса:

- чтение Excel
- обработка портфеля проектов
- анализ финансов
- работа с CRM
- интеграция с API

Именно здесь происходит переход от механического перебора строк к структурированному моделированию данных

Далее мы рассмотрим практический кейс с вложенными словарями и списками, где:

- внутри списка объектов будут вложенные словари
- внутри словарей будут списки
- структура станет многоуровневой
- появится полноценная модель бизнес-системы

Мы разберём:

- как проектировать такую структуру
- как правильно проходить по вложенным уровням
- как строить аналитику на нескольких уровнях
- как не запутаться в сложных данных

Это будет переход к продвинутой архитектуре RPA.

ПРИМЕР. КЕЙС: РАЗРАБОТКА АНАЛИТИЧЕСКОЙ СИСТЕМЫ «УЧЕБНЫЙ ЦЕНТР»

Контекст задачи

Вы работаете над проектом для виртуального учебного центра, который активно развивается: открываются новые курсы, подключаются преподаватели и регистрируются студенты. Руководство хочет иметь прозрачную и удобную систему аналитики, чтобы принимать управленческие решения.

Ваша задача – создать систему сбора и анализа данных с помощью Puzzle RPA, которая будет:

- Хранить данные о курсах, преподавателях и студентах.
- Выполнять аналитику успеваемости студентов и курсов.
- Формировать управленческие отчёты для директора учебного центра.

Перед тем как приступать к заданиям, важно осознать несколько ключевых моментов:

1.Структура данных

Все данные будут храниться в виде вложенных словарей и списков Python, например:

```
education_center = {
    "courses": [
        {
            "name": "Название курса",
            "teachers": [{"name": "Имя", "email": "email"}],
            "students": [
                {"name": "Имя", "email": "email", "grades": [80, 75, 90]}
            ]
        }
    ]
}
```

`courses` – список всех курсов. Каждый курс содержит преподавателей (`teachers`) и студентов (`students`). У студентов есть оценки (`grades`), которые будут использоваться для аналитики.

2.Объём данных

Для практики используем данные, сгенерированные через нейросеть (DeepSeek) или вручную. Минимальные требования:

- ≥5 курсов
- В каждом курсе 2 преподавателя
- В каждом курсе 10–15 студентов с 3–5 оценками
- ≥25 уникальных студентов
- Некоторые студенты могут быть записаны на несколько курсов

После подготовки данных мы будем:

- Проверять корректность данных
- Вычислять средние баллы студентов и курсов
- Определять лучших студентов и курсы
- Формировать группы риска и сегменты студентов
- Анализировать нагрузку преподавателей
- Выявлять студентов, обучающихся на нескольких курсах

В ходе задания будем использовать следующие блоки Puzzle RPA:

- присвоение переменных,
- логические проверки,
- циклы,
- вычисление средних значений,
- создание списков и словарей.

Каждый блок имеет назначение и «идёт» в отчёт или является промежуточным.

Важно понимать, что промежуточные переменные (например, `merged_students`) нужны для расчётов, а в итоговый отчёт попадают только ключевые показатели (`TotalCourses`, `BestCourse`, `TopStudent` и т.д.).

Как выглядит общий процесс работы

- Подготовка данных
- Проверка наличия курсов
- Подсчёт общего числа курсов и студентов
- Объединение списков студентов для проверки пересечения
- Аналитика успеваемости
- Средний балл каждого студента
- Средний балл по курсам
- Определение лучшего курса и лучшего студента
- Выявление студентов с низкой успеваемостью
- Сегментация и расширенная аналитика

- Формирование групп по уровню успеваемости
- Подсчёт нагрузки преподавателей
- Выявление студентов, обучающихся на нескольких курсах
- Создание карты «студент → курсы»
- Формирование итогового отчёта

Все ключевые показатели собираются в словари: `education_basic_report` и `education_extended_report`



Первичная подготовка исходных данных

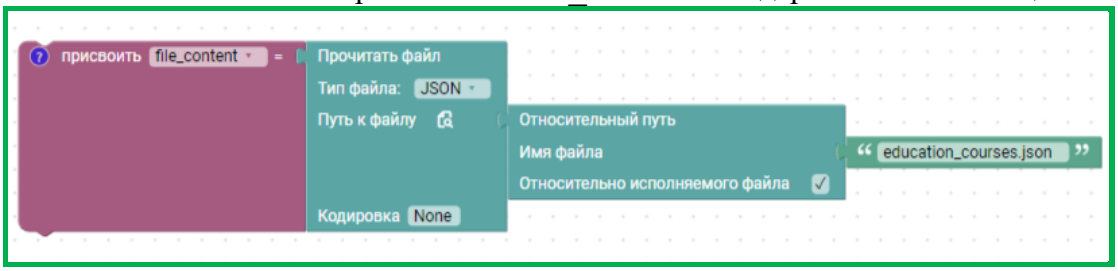
Перед тем как начать аналитику, нам нужно создать файл с исходными данными и положить его в проект Puzzle RPA:

- Создаём JSON-файл. Имя файла: `education_center.json`
- Помещаем JSON-файл в папку Resources проекта Puzzle RPA. Этот файл будет использоваться роботам для считывания данных и последующего анализа.

Шаг 1. Прочитать JSON - файл

- Блок: Прочитать файл JSON
- Относительный путь: `education_center.json`
- Сохранить в переменную: `file_content`
- Тип переменной: Словарь (`Dictionary`)

После выполнения блока переменная `file_content` содержит весь JSON целиком.

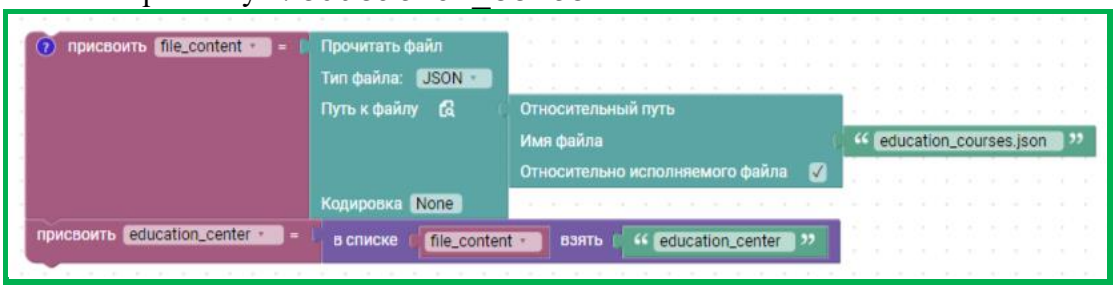


Шаг 2. Взять education_center из file_content

Теперь нужно получить вложенный словарь "`education_center`".

- Блок: В словаре взять
- Словарь: `file_content`
- Ключ:"`education_center`"

- Сохранить в переменную: `education_center`



Мы достали вложенный словарь и сохранили его отдельно.

Что делает: загружает все данные о курсах, преподавателях и студентах в память робота, чтобы дальше можно было проводить аналитику.

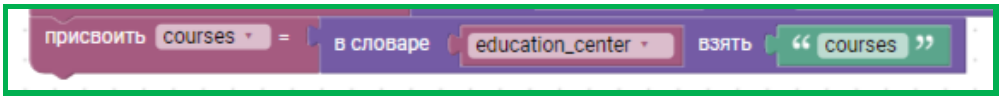
Таким образом, на этом этапе мы подготовили исходные данные и сделали их доступными для всех последующих блоков анализа и формирования отчётов.



Шаг 3. Взять список курсов

Теперь берём список "courses".

- Блок: В словаре взять
- Словарь: `education_center`
- Ключ: "`courses`"



- Сохранить в переменную: `courses`
- Тип переменной: Список (`List`)

Теперь: `courses` – это список словарей курсов.

Очень частая ошибка – пытаться сразу брать "`courses`" из `file_content`.

Но структура вложенная:

- Сначала берём "`education_center`"
- Потом берём "`courses`"
- Работаем слой за слоем.
- Проверка корректности данных перед аналитикой
- Переменная: `is_courses_empty`

Вторичная подготовка исходных данных

Перед тем как запускать аналитику, необходимо убедиться, что список курсов не пустой.

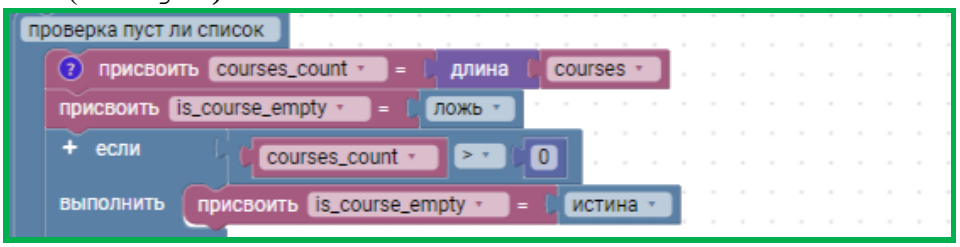
Если курсов нет – дальнейшие расчёты (средние значения, лучший курс и т.д.) выполнять невозможно.

Шаг 1. Убедиться, что список курсов получен

Предполагается, что ранее уже был выполнен блок «В словаре взять» и получена переменная `courses` типа `Список`.

Шаг 2. Определить количество курсов

- Блок: Присвоить переменную
- Имя переменной: `courses_count`
- Тип: Числовой (`Integer`)



Что делаем: Используем функцию подсчёта длины списка.

Зачем это нужно: Чтобы проверить, равен ли размер списка нулю.

Шаг 3. Создать логическую проверку

- Блок: Присвоить переменную
- Имя переменной: `is_courses_empty`
- Тип: Логический (`Boolean`)

Что делаем: Сравниваем количество курсов с нулём.

Результат: Если значение `True` – список курсов пуст.

Если `False` – данные есть и можно продолжать анализ.

Почему это важно:

- Это защита от ошибки выполнения.
- Это контроль корректности входных данных.
- Это обязательная проверка перед запуском циклов.
- Это логическая переменная, которая используется только для контроля выполнения и не входит в итоговый отчёт.

`is_courses_empty` – это:

- не аналитическая переменная,
- не показатель отчёта,
- а контрольная логическая переменная,
- которая проверяет корректность данных перед вычислениями

Объединение студентов двух выбранных курсов

Перед аналитикой может понадобиться проверить, пересекаются ли студенты между разными курсами.

Для этого создаётся вспомогательная переменная `merged_students`.

Цель переменной

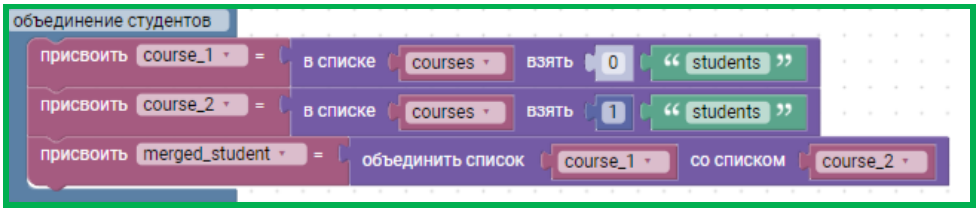
- Объединить списки студентов двух выбранных курсов.
- Подготовить данные для анализа пересечений.
- Используется только для расчётов.
- В итоговый отчёт не входит.

Главное правило работы со вложенными структурами:

- Если структура выглядит так:
  ✓ courses – это список
  ✓ каждый элемент списка – это словарь курса
  ✓ внутри словаря есть ключ "students"
- То порядок действий всегда такой:
  ✓ Сначала берём индекс (из списка), потом берём ключ (из словаря).

Логика реализации в Puzzle RPA

- Шаг 1. Взять первый курс (работаем со списком)
  ✓ Блок: В списке взять
  ✓ Список: `courses_list`
  ✓ Индекс: нужный номер курса
  ✓ Ключ: `"students"`
  ✓ Сохраняем в переменную `course_1`



Мы получили список студентов первого курса.

- Шаг 2. Повторить для второго курса

  В списке взять → берём второй курс по индексу, берём ключ "students"
- Шаг 3. Объединить списки

  ✓ Блок: Присвоить переменную

  ✓ Имя: merged_students

  ✓ Объединяем два списка студентов

  Теперь merged_students содержит студентов обоих курсов.

  Почему порядок важен? Частая ошибка:
- Пытаться взять ключ "students" сразу из списка courses_list.
- Но список не содержит ключей – у него есть индексы.

  Правильная логика: Список → индекс → словарь → ключ → значение

  merged_students – это:
- вспомогательная переменная;
- используется для анализа пересечений;
- не включается в итоговый отчёт.

Подсчёт уникальных студентов

После того, как мы объединили два списка студентов в переменную merged_students, в этом списке могут быть дубликаты.

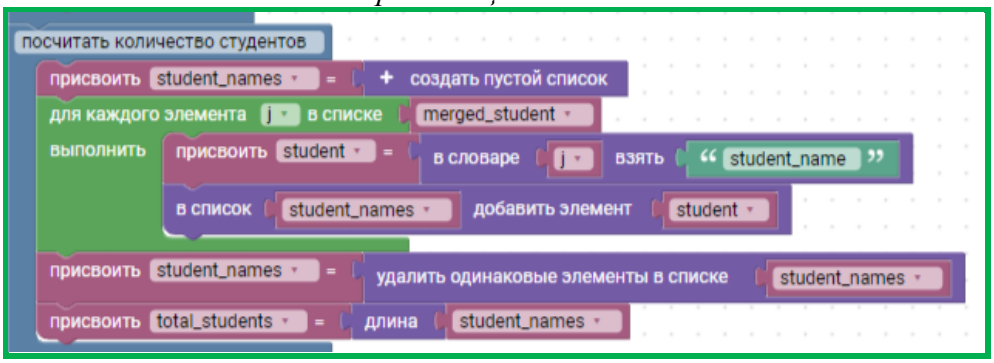
Это происходит потому, что один и тот же студент может обучаться на нескольких курсах.

Наша цель – получить список уникальных имён студентов, чтобы корректно определить их общее количество.

Этот показатель:

- используется в отчёте;
- входит как TotalStudents;
- является управленческой метрикой.

Логика реализации в Puzzle RPA:



- Шаг 1. Создать пустой список для имён

  ✓ Блок: Присвоить переменную

  ✓ Имя переменной: student_names

  ✓ Значение: пустой список

  Этот список будет содержать только имена студентов.
- Шаг 2. Пройти циклом по merged_students

  ✓ Блок: Цикл «Для каждого элемента»

  ✓ Список: merged_students

  ✓ Текущий элемент: j

  Мы перебираем словари студентов.
- Шаг 3. Внутри цикла взять имя

  ✓ Блок: В словаре взять

✓ Словарь: `j`
✓ Ключ: `"student_name"`
✓ Сохранить во временную переменную `student`

- Шаг 4. Добавить имя в список
  ✓ Блок: В список добавить элемент
  ✓ Список: `student_names`
  ✓ Элемент: имя студента (переменная `student`)
  После завершения цикла в списке могут быть повторы.
- Шаг 5. Удалить дубликаты
  ✓ Блок: Присвоить переменную
  ✓ Имя переменной: обновить переменную student_names
  ✓ Используем блок удалить одинаковые элементы в списке
  Это автоматически удаляет повторяющиеся значения.

Если не убрать дубликаты: один студент, обучающийся на двух курсах, будет посчитан дважды, и показатель `total_students` будет некорректным.

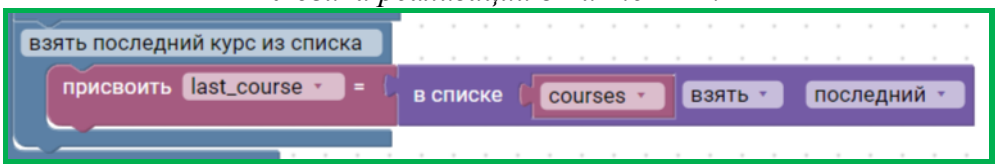
Далее мы будем использовать данные в отчёте как переменная `"total_students"`
При формировании итогового словаря: в поле передаётся: длина списка student_names



Получение последнего курса из списка

В учебном центре новые программы добавляются в конец списка курсов.
Поэтому последний элемент списка – это самая новая программа.
Нам нужно получить словарь последнего курса, чтобы:
✓ отдельно проанализировать новую программу;
✓ посмотреть её студентов;
✓ оценить первые результаты обучения.
Эта переменная используется для анализа, но в итоговый отчёт напрямую не включается.

Логика реализации в Puzzle RPA:



В Puzzle RPA не обязательно вычислять индекс вручную.
Можно использовать встроенную возможность взять последний элемент списка.

- Блок: В списке взять
- В настройках блока выбираем режим: «Последний элемент»
- Настройки блока: Список: `courses`
- Режим выбора: Последний
- Сохранить в переменную: `last_course`

Что происходит:

- Блок автоматически берёт последний элемент списка.
- Не нужно считать длину списка.
- Не нужно вычитать единицу.
- Робот сам корректно определяет последний индекс.

Важно помнить:

- Мы работаем со списком → используем блок «В списке взять».
- Выбираем режим «Последний».
- Результат – словарь одного курса.

Аналитика успеваемости

Средний балл каждого студента

Для каждого студента необходимо:

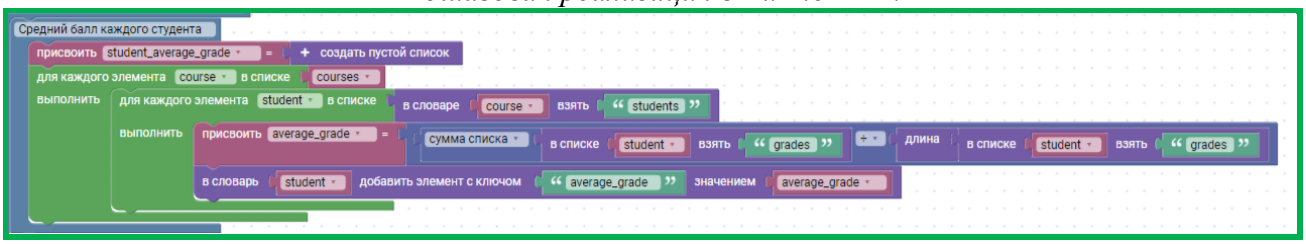
- рассчитать средний балл;
- добавить его как новый ключ в словарь студента;

использовать этот показатель далее для:

- поиска лучшего студента,
- сегментации,
- выявления группы риска.

Это индивидуальный аналитический показатель.

Пошаговая реализация в Puzzle RPA:



- Шаг 1. Создать пустой список
  - ✓ Блок: Присвоить переменную
  - ✓ Имя переменной: `student_averages_grade`
  - ✓ Значение: пустой список

  Важно: на этом этапе список не содержит элементов.
- Шаг 2. Пройти по всем курсам
  - ✓ Блок: Цикл «Для каждого элемента»
  - ✓ Список: `courses`
  - ✓ Текущий элемент: `course`
  - ✓ Мы перебираем каждый курс.
- Шаг 3. Пройти по студентам (Внутренний цикл)
  - ✓ Блок: Цикл «Для каждого элемента»
  - ✓ Список: в словаре course взять ключ «`students`»
  - ✓ Текущий элемент: `student`

  Теперь мы перебираем студентов конкретного курса.
- Шаг 4. Рассчитать среднее значение
  - ✓ Блок: Присвоить переменную
  - ✓ Имя: `average_grade`
  - ✓ Тип: Числовой (`Float`)
  - ✓ Используем:
    - – сумму элементов списка
    - – деление на количество элементов

    Это и есть средний балл студента.
  - ✓ Внутри расчёта берем оценки студента через блок «в списке взять»
    - – Словарь: `student`
    - – Ключ: "`grades`"
- Шаг 6. Добавить новый ключ в словарь студента
- Блок: В словарь добавить элемент с ключом значением
- Словарь: student
- Ключ: "`average_grade`"
- Значение: `average_grade`

Что происходит в результате:

- Каждый студент теперь имеет структуру:
  - – `name`
  - – `email`
  - – `grades`
  - – `average_grade` ← новый аналитический показатель

Важно:

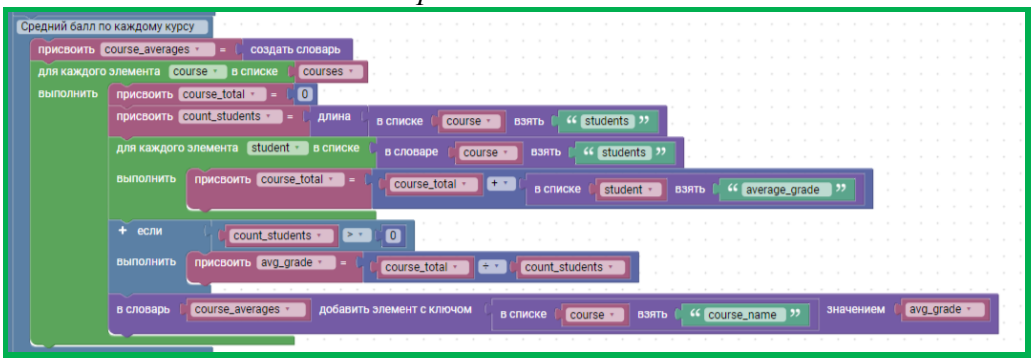
- Это вложенный цикл (курс → студенты).
- Мы не создаём новый список студентов.
- Мы модифицируем существующие словари.
- Добавление нового ключа не разрушает структуру – оно её расширяет.

Средний балл по каждому курсу

Наша цель:

- Определить, насколько эффективно проходят обучение разные курсы.
- Получить показатель, который можно включить в отчёт как `CourseAverages`.
- Используется для сравнения курсов между собой.

Логика работы в Puzzle RPA:



- Шаг 1. Создать пустой словарь для хранения средних баллов
  - ✓ Блок: Присвоить переменную
  - ✓ Имя переменной: `course_averages`
  - ✓ Тип: Словарь (`Dictionary`)
  - ✓ Значение: пустой словарь

  Этот словарь будет хранить пары: `"Название курса" → средний балл`
- Шаг 2. Пройти циклом по всем курсам
  - ✓ Блок: Цикл «Для каждого элемента»
  - ✓ Список: courses
  - ✓ Текущий элемент: course
  - ✓ Мы перебираем каждый курс, чтобы обработать его студентов.
- Шаг 3. Чтобы вычислить средний балл курса, нужны две ключевые величины:
  - ✓ Сумма всех оценок студентов курса → переменная `course_total`
  - ✓ Количество студентов курса → переменная `count_students`
  - ✓ 1. Инициализируем переменную для суммы
    - – Блок: Присвоить переменную
    - – Имя переменной: `course_total`
    - – Тип: Числовой (`Integer` или `Float`)
    - – Значение: 0

    Эта переменная будет накапливать сумму всех средних баллов студентов курса.
  - ✓ 2. Определяем количество студентов курса
    - – Блок: В списке взять → длина списка студентов
    - – Сначала: берем список студентов курса
    - – Блок: В словаре взять
    - – Словарь: `course`
    - – Ключ: `"students"`
    - – Затем: присваиваем переменной `count_students`
    - – Блок: Присвоить переменную
    - – Имя переменной: `count_students`
    - – Тип: Числовой (`Integer`)

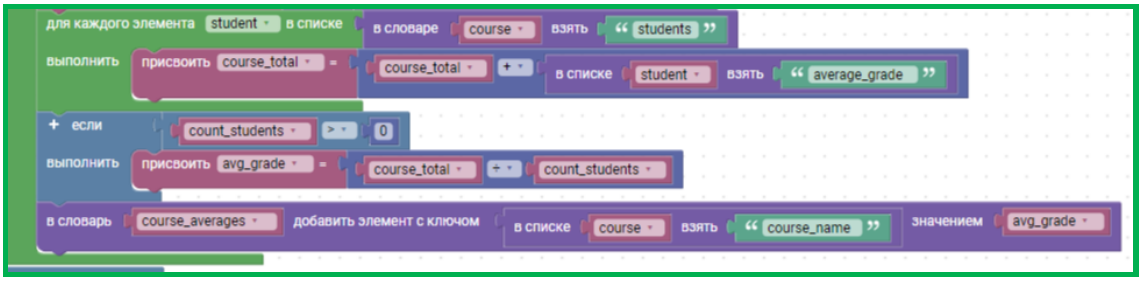
    Важно: сначала берём список студентов через ключ, потом считаем его длину.

    Теперь, когда у нас есть:

    – `course_total = 0` – сумма всех средних баллов студентов курса

    – `count_students` = длина списка студентов курса

    можно проходить по студентам и накапливать оценки.



- Шаг 4. Цикл по студентам курса
  - ✓ Блок: Цикл «Для каждого элемента»
  - ✓ Список: в словаре «`course`» взять ключ «`students`»
  - ✓ Текущий элемент: `student`
  - ✓ Проходим каждого студента, чтобы взять его средний балл.
- Шаг 3. Накопить в `course_total`
  - ✓ Блок: Присвоить переменную
  - ✓ Имя переменной: `course_total`
  - ✓ Тип: Числовой (`Float`)
  - ✓ Значение: `course_total + student_average` (взять средний балл студента: в словаре «student» взять ключ "`average_grade`"
    Здесь используется уже рассчитанный ранее средний балл каждого студента.
    Таким образом мы суммируем средние баллы всех студентов курса.
- Шаг 4. После цикла – вычислить средний балл курса
  - ✓ Блок: Условие `если «count_students» > 0`
  - ✓ Выполнить Блок: Присвоить переменную
  - ✓ Имя переменной: `avg_grade`
  - ✓ Тип: Числовой (`Float`)
  - ✓ Значение: `course_total / count_students`
    Результат – средний балл курса, учитывающий всех студентов.
- Шаг 5. Сохранить средний балл в словарь `course_averages`
  - ✓ Блок: В словарь добавить элемент с ключом значением
  - ✓ Словарь: `course_averages`
  - ✓ Ключ: название курса (`course["name"]`)
  - ✓ Значение: `avg_grade`

В итоге в словаре `course_averages` накапливаются пары: `"Название курса" → средний балл`

Важно:

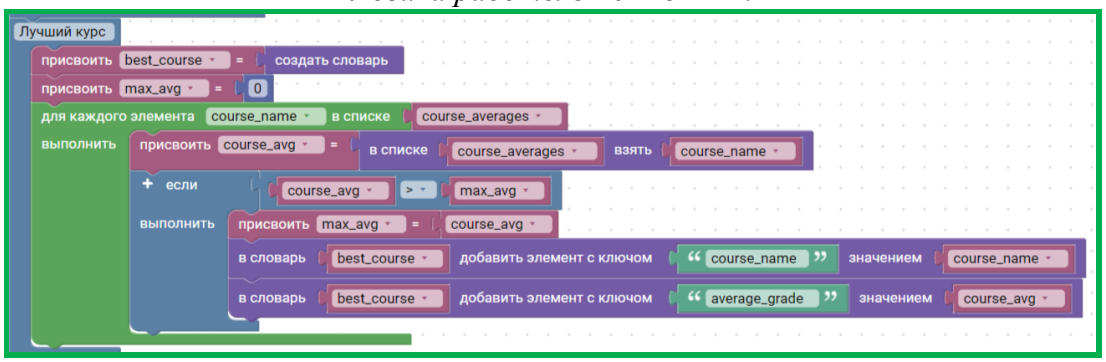
- Сначала список → потом ключ
- Берём `students_list` через ключ "`students`", затем работаем с элементами списка студентов
- Накопление через переменную: `course_total` постепенно увеличивается на каждого студента
- Вычисление среднего. Делим сумму на количество студентов (`count_students`)
- Словарь для итогов. `course_averages` удобно использовать для формирования отчёта и сравнения курсов

Лучший курс

Цель:

- Определить курс с наивысшим средним баллом.
- Сформировать отдельный словарь с ключами: "`name`" – название курса; "`average_grade`" – средний балл курса

Логика работы в Puzzle RPA:



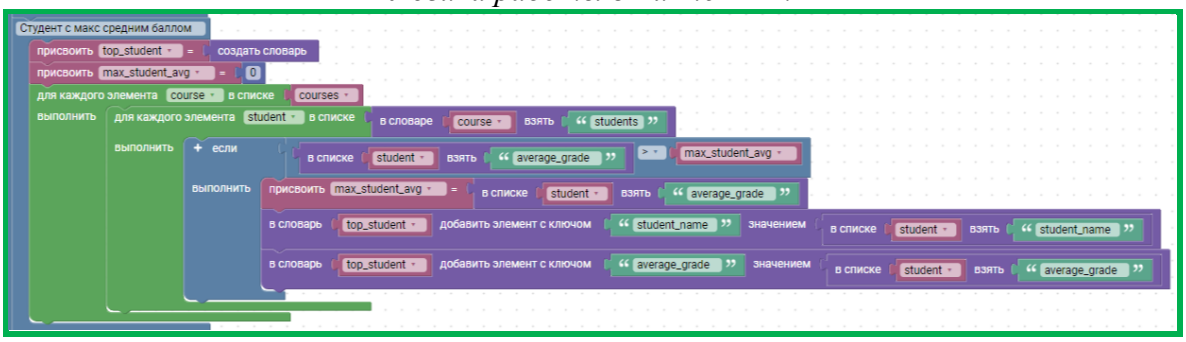
- Шаг 1. Создать пустой словарь для лучшего курса
  ✓ Блок: Присвоить переменную
  ✓ Имя переменной: `best_course`
  ✓ Значение: пустой словарь
  Сейчас мы просто резервируем место для записи данных о лучшем курсе.
- Шаг 2. Создать переменную для максимального среднего
  ✓ Блок: Присвоить переменную
  ✓ Имя переменной: `max_average`
  ✓ Тип: Числовой (`Float`)
  ✓ Значение: 0
  Эта переменная будет хранить наивысшее среднее среди всех курсов.
- Шаг 3. Цикл по словарю `course_averages`
  ✓ Блок: Цикл «Для каждого элемента»
  ✓ Словарь: `course_averages`
  ✓ Текущий элемент: `course_name`
  Этот цикл перебирает все названия курсов.
- Шаг 4. Взять средний балл курса
  ✓ Блок: В списке взять
  ✓ Словарь: `course_averages`
  ✓ Ключ: `course_name`
  ✓ Сохранить в переменную: `course_avg`
  ✓ Тип переменной: Числовой (`Float`)
  Теперь у нас есть средний балл конкретного курса.
- Шаг 5. Сравнить с текущим максимумом
  ✓ Блок: Условие
  ✓ Условие: `course_avg > max_average`
  ✓ Если средний балл курса выше текущего максимума:
  ✓ Обновляем `max_average = course_avg`
- Шаг 6. Обновить словарь `best_course`
  ✓ Блок: В словарь добавить элемент с ключом значением
  ✓ Словарь: `best_course`
  ✓ Ключи и значения:
  – `"course_name " → course_name`
  – `"average_grade" → course_avg`
  Теперь `best_course` хранит самый успешный курс на данный момент.
- Шаг 7. После завершения цикла в переменной `best_course` остаётся один словарь. Этот словарь можно включить в итоговый отчёт.

Студент с максимальным средним баллом

Цель:

- Найти студента с наивысшим средним баллом среди всех студентов всех курсов.
- Сформировать отдельный словарь с ключами:
  ✓ `"name"` – имя студента
  ✓ `"average_grade"` – средний балл
- Включить в отчёт, чтобы показать рейтинг лучших студентов.

Логика работы в Puzzle RPA:



- Шаг 1. Создать пустой словарь для результата
  - ✓ Блок: Присвоить переменную
  - ✓ Имя переменной: `top_student`
  - ✓ Значение: пустой словарь

  Резервируем место для записи данных о лучшем студенте.
- Шаг 2. Создать переменную для максимального среднего
  - ✓ Блок: Присвоить переменную
  - ✓ Имя переменной: `max_student_avg`
  - ✓ Тип: Числовой (`Float`)
  - ✓ Значение: `0`

  Эта переменная будет хранить наивысший средний балл среди всех студентов.
- Шаг 3. Цикл по всем курсам
  - ✓ Блок: Цикл «Для каждого элемента»
  - ✓ Список: `courses`
  - ✓ Текущий элемент: `course`
  - ✓ Проходим каждый курс, чтобы обработать студентов.
- Шаг 4. Цикл по студентам курса
  - ✓ Блок: Цикл «Для каждого элемента»
  - ✓ Список: В словаре «`course`» взять ключ "`students`"
  - ✓ Текущий элемент: `student`
  - ✓ Теперь перебираем каждого студента.
- Шаг 5. Добавить условие
  - ✓ Блок Если в списке» student» взять ключ"`average_grade`"
  - ✓ Сравнить с текущим максимумом > `max_student_average`
  - ✓ Если средний балл больше текущего максимума:
  - ✓ Обновляем `max_student_avg = в списке» student» взять ключ "average_grade"`
- Шаг 8. Обновить словарь `top_student`
  - ✓ Блок: В словарь добавить элемент с ключом значением
  - ✓ Словарь: `top_student`
  - ✓ Ключи и значения:
    - "`student name`" → `student["name"]`
    - "`average_grade`" → `average_grade`

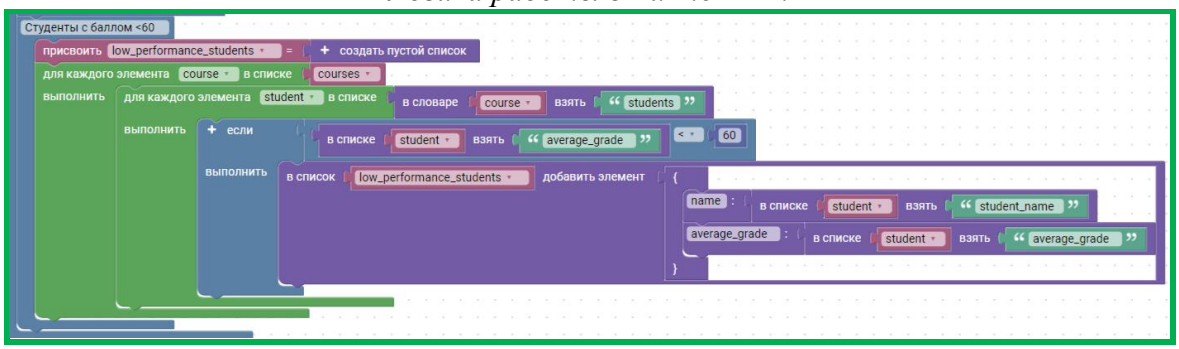
  В результате словарь `top_student` всегда содержит студента с наивысшим средним баллом на данный момент.
- Шаг 9. После завершения всех циклов в `top_student` хранится один словарь. Можно включать его в итоговый отчёт.

Студенты с баллом < 60

Цель:

- Выявить студентов, у которых средний балл меньше 60.
- Сформировать отдельный список или словарь для группы риска.
- Включить в итоговый отчёт.

Логика работы в Puzzle RPA:



- Шаг 1. Создать пустой список для группы риска
  - ✓ Блок: Присвоить переменную
  - ✓ Имя переменной: `low_performance_students`
  - ✓ Значение: пустой список
  - ✓ В список будем добавлять словари каждого студента с низкой успеваемостью.
- Шаг 2. Цикл по всем курсам
  - ✓ Блок: Цикл «Для каждого элемента»
  - ✓ Список: `courses`
  - ✓ Текущий элемент: course
  - ✓ Перебираем каждый курс, чтобы обработать студентов.
- Шаг 3. Цикл по студентам курса
  - ✓ Блок: Цикл «Для каждого элемента»
  - ✓ Список: В словаре `course` взять ключ `"students"`
  - ✓ Текущий элемент: `student`
  - ✓ Теперь перебираем каждого студента.
- Шаг 4. Проверить условие низкой успеваемости
- Блок Если в списке «`student`» взять ключ `"average_grade" < 60`
- Если условие выполняется: Создаём словарь в списке для студента:
- Блок: В список `low_performance_students`
- Добавить элемент словарь с ключами и значениями:
  - ✓ `"name" → student["name"]`
  - ✓ `"average_grade" → student["average_grade "]`

  Таким образом в списке будут только студенты с баллом <60.
- Шаг 5. После завершения всех циклов

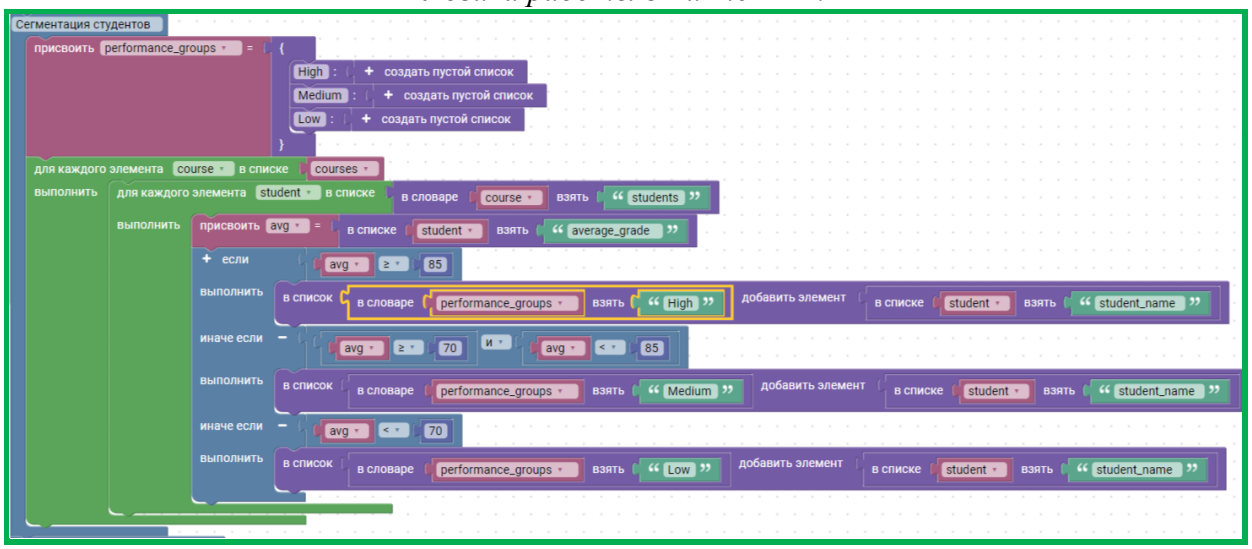
В переменной `low_performance_students` хранится список словарей. Этот список можно включить в итоговый отчёт.

Часть 2. Сегментация студентов

Цель:

- Разделить студентов на группы по среднему баллу:
  - ✓ `High (≥85)`
  - ✓ `Medium (70-84)`
  - ✓ `Low (<70)`
- Сформировать словарь `performance_groups` для аналитики и отчёта.

Логика работы в Puzzle RPA:



- Шаг 1. Создать пустой словарь для групп
- Блок: Присвоить переменную
- Имя переменной: `performance_groups`
- Тип: Словарь (`Dictionary`)
- Значение:

```
{
  "High": [],
  "Medium": [],
  "Low": []
}
```

В словари будут добавляться имена студентов по уровням подготовки.

- Шаг 2. Цикл по всем курсам
  - ✓ Блок: Цикл «Для каждого элемента»
  - ✓ Список: `courses`
  - ✓ Текущий элемент: course
  - ✓ Перебираем все курсы, чтобы обработать студентов.
- Шаг 3. Цикл по студентам курса
  - ✓ Блок: Цикл «Для каждого элемента»
  - ✓ Список: В словаре `course` взять ключ `"students"`
  - ✓ Текущий элемент: `student`
  - ✓ Теперь перебираем каждого студента.
- Шаг 4. Взять средний балл студента
  - ✓ Блок: В списке student взять ключ `"average_grade"`
  - ✓ Сохранить во временную переменную: `avg`
  - ✓ Тип: Числовой (`Float`)
- Шаг 5. Определить группу по успеваемости
  - ✓ Блок: Условие
  - ✓ Если `avg >= 85` → добавить имя в `performance_groups["High"]`

    Важно: как правильно добавлять студентов в `performance_groups`. Основная логика
  - ✓ Словарь: `performance_groups`
  - ✓ Ключи: `"High"`, `"Medium"`, `"Low"`
  - ✓ Значения: списки имён студентов
  - ✓ Элемент списка: `student["name"]`

    Важно: сначала через ключ словаря берём список, потом добавляем в него имя студента.
  - ✓ Иначе если `70 <= avg < 85` → добавить в `performance_groups["Medium"]`
  - ✓ Если `avg < 70` добавить в `performance_groups["Low"]`

    В результате каждый студент будет распределён по одной из трёх групп.
- Шаг 6. После завершения всех циклов

В словаре `performance_groups` будут списки студентов по уровням подготовки. Этот словарь включается в итоговый отчёт для стратегического планирования.

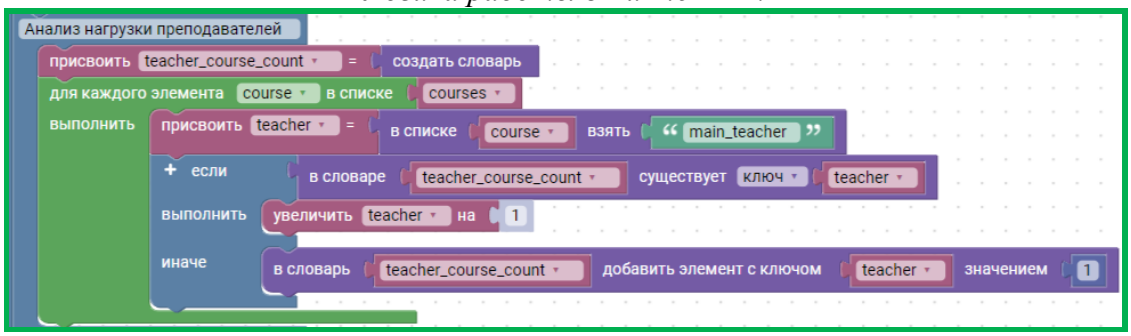
Анализ нагрузки преподавателей

Цель:

- Определить, сколько курсов ведёт каждый преподаватель.
- Сформировать словарь `teacher_course_count` для включения в отчёт.

Помогает понять, равномерно ли распределена нагрузка и оптимизировать планирование.

Логика работы в Puzzle RPA:



- Шаг 1. Создать пустой словарь для подсчёта
  - ✓ Блок: Присвоить переменную
  - ✓ Имя переменной: `teacher_course_count`
  - ✓ Значение: пустой словарь
  - ✓ В словарь будем добавлять пары: `"Имя преподавателя" → количество курсов`
- Шаг 2. Цикл по всем курсам
  - ✓ Блок: Цикл «Для каждого элемента»
  - ✓ Список: `courses`
  - ✓ Текущий элемент: `course`
  - ✓ Проходим каждый курс, чтобы обработать преподавателей.
- Шаг 3. Взять список преподавателей курса
  - ✓ Блок: В списке взять
  - ✓ Список: `course`
  - ✓ Ключ: `"main_teacher"`
  - ✓ Сохранить в переменную: `teacher`
  - ✓ Тип: Список (`List`)

Шаг 4. Условие: существует ли ключ в словаре

- Блок: Условие
- Если Блок: в словаре «`teacher_course_count`» существует ключ `"teacher"`
- Проверяем, есть ли уже этот преподаватель в словаре.
- Если ключ существует (`True`)
- Значение: `"teacher"` увеличить на 1
- Увеличиваем количество курсов преподавателя на 1.
- Если ключ не существует (`False`)
- Блок: В словарь добавить элемент с ключом значением
- Словарь: `teacher_course_count`
  - ✓ Ключ: `teacher`
  - ✓ Значение: 1

Таким образом, для каждого преподавателя подсчитывается количество курсов, которые он ведёт.

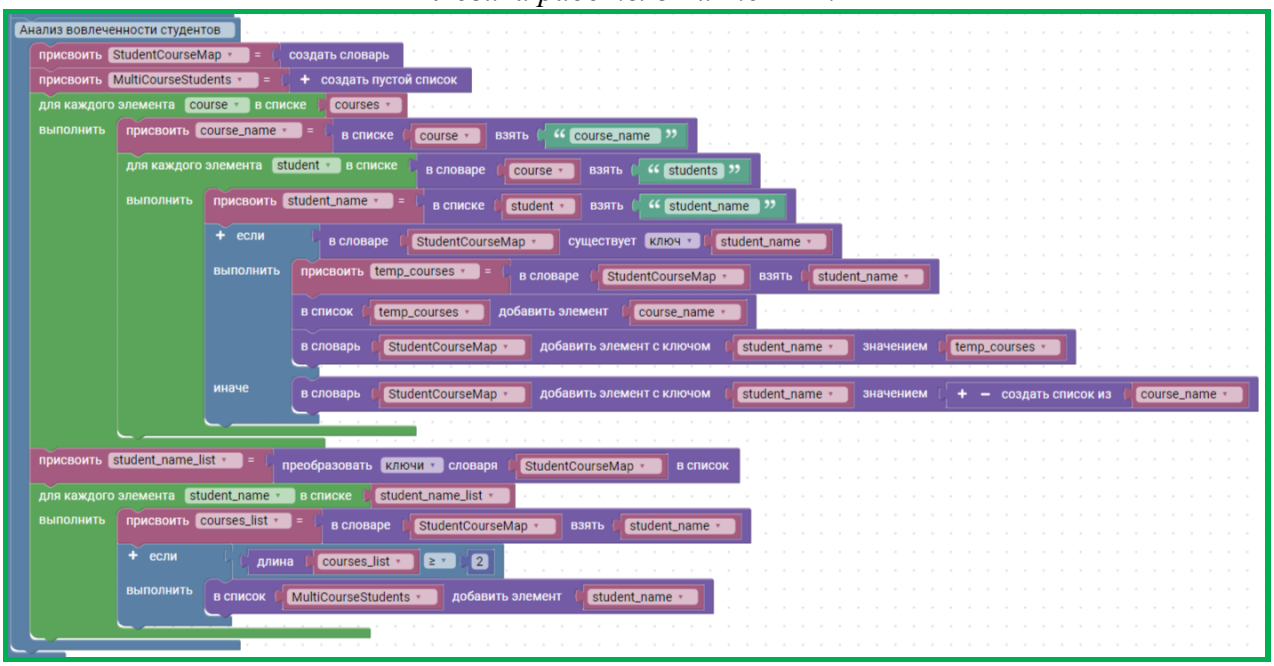
- Шаг 5. После завершения всех циклов

В словаре `teacher_course_count` хранится словарь. Этот словарь можно включить в итоговый отчёт.

Анализ вовлечённости студентов

Цель: Проанализировать вовлечённость студентов центра, чтобы выявить тех, кто обучается на нескольких курсах, и получить полное распределение студентов по программам.

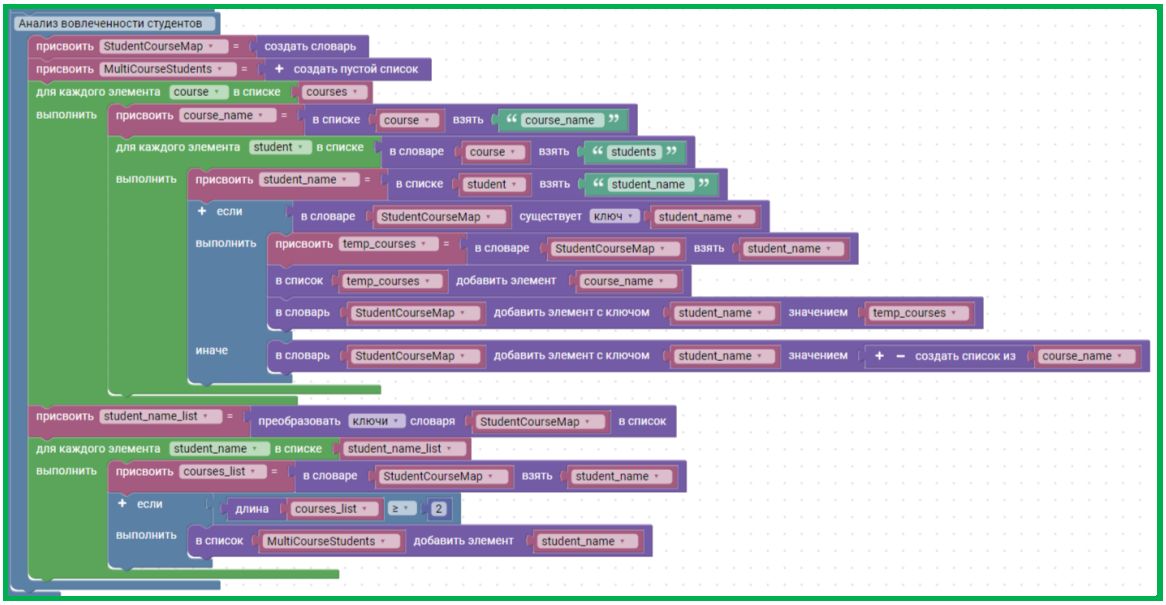
Логика работы в Puzzle RPA:



- Шаг 1. Создание пустых переменных
  - ✓ Блок: Присвоить значение переменной
  - ✓ Имя: `StudentCourseMap`
  - ✓ Значение: `{}`
  - ✓ Тип: Словарь

    Зачем: сюда будем складывать студентов и их курсы
  - ✓ Блок: Присвоить значение переменной
  - ✓ Имя: `MultiCourseStudents`
  - ✓ Значение: `[]`
  - ✓ Тип: Список

    Зачем: сюда добавим студентов с несколькими курсами
- Шаг 2. Цикл по всем курсам
  - ✓ Блок: Цикл «Для каждого элемента»
  - ✓ Список: `courses`
  - ✓ Текущий элемент: `course`
- Шаг 3. Извлечь название курса
  - ✓ Блок: Присвоить значение переменной
  - ✓ Имя: `course_name`
  - ✓ Значение: `course['course_name']`
- Шаг 4. Цикл по студентам курса
  - ✓ Блок: Цикл «Для каждого элемента»
  - ✓ Список: В словаре `course` взять ключ `"students"`
  - ✓ Текущий элемент: `student`
  - ✓ Теперь перебираем каждого студента.
- Шаг 5. Извлечь имя студента
  - ✓ Блок: Присвоить значение переменной
  - ✓ Имя: `student_name`
  - ✓ Значение: `student['student_name']`



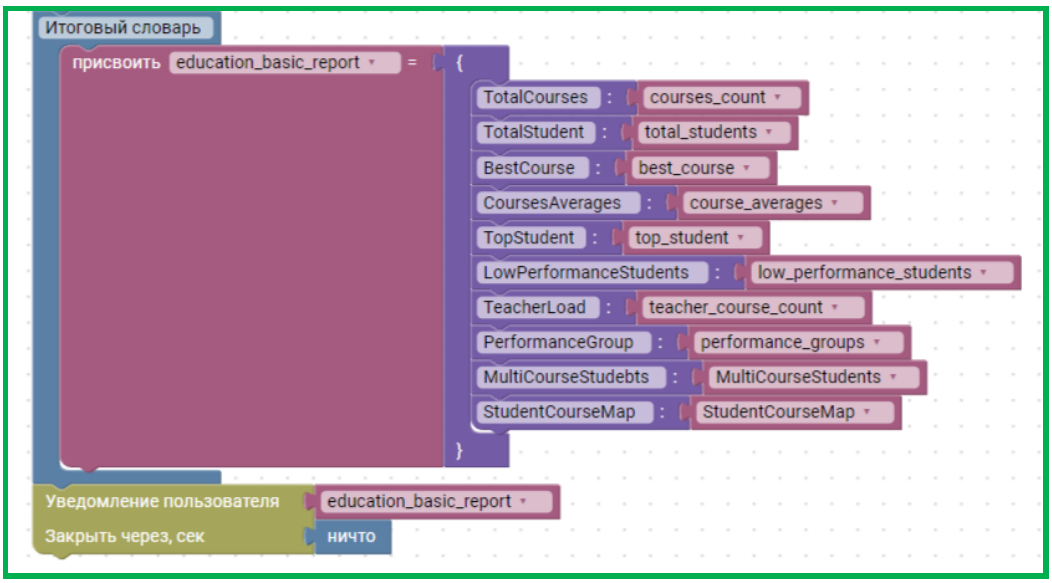
- Шаг 6. Проверить, есть ли студент в словаре
  - ✓ Блок: Если – выполнить
  - ✓ Условие: ключ `student_name` уже есть в `StudentCourseMap`
  - ✓ Если студент есть: Блок присвоить переменную
  - ✓ Имя: `temp_courses`
  - ✓ Значение: `StudentCourseMap[student_name]`
  - ✓ Блок: Добавить элемент в список
  - ✓ Список: `temp_courses`
  - ✓ Элемент: `course_name`
  - ✓ Блок: Добавить элемент в словарь
  - ✓ Словарь: `StudentCourseMap`
  - ✓ Ключ: `student_name`
  - ✓ Значение: `temp_courses`
  - ✓ Если студента нет: Блок добавить элемент в словарь
  - ✓ Словарь: `StudentCourseMap`
    - – Ключ: `student_name`
    - – Значение: создать список `course_name`
- Шаг 7. Преобразовать словарь в список студентов
  - ✓ Блок: Присвоить значение переменной
  - ✓ Имя: `student_name_list`
  - ✓ Значение: Преобразовать ключи словаря `StudentCourseMap` в список
    Зачем: получить все имена студентов для дальнейшего анализа
- Шаг 8. Определить студентов на ≥2 курсах
  - ✓ Блок: Цикл для каждого элемента
  - ✓ Список: `student_name_list`
  - ✓ Текущий элемент: `student_name`
  - ✓ Извлечь список курсов студента
- Блок: Присвоить значение переменной
- Имя: `courses_list`
- Значение: `StudentCourseMap [student_name]`
- Шаг 9. Проверить количество курсов
- Блок: Если – выполнить
- Условие: `len(courses_list) >= 2`
- Добавить в список `MultiCourseStudents`
- Блок: Добавить элемент в список
- Список: `MultiCourseStudents`
- Элемент: `student_name`

- Результат после выполнения всех блоков:
  ✓ `StudentCourseMap` → словарь всех студентов и их курсов
  ✓ `MultiCourseStudents` → список студентов, которые учатся на ≥2 курсах

Итоговый словарь

- Блок: Присвоить значение переменной
- Имя: `education_basic_report`
- Тип: Словарь

Описание: собирает все ключевые показатели центра в одной структуре для отчёта и уведомления пользователя.



Структура словаря и пояснения:

Ключ	Содержание	Описание
`TotalCourses`	`courses_count`	Общее количество курсов в центре
`TotalStudent`	`total_students`	Общее количество студентов
`BestCourse`	`best_course`	Курс с лучшими результатами (по среднему баллу)
`CoursesAverages`	`course_averages`	Словарь средних баллов по каждому курсу
`TopStudent`	`top_student`	Студент с наивысшими показателями успеваемости
`LowPerformanceStudents`	`low_performance_students`	Список студентов с низкой успеваемостью
`TeacherLoad`	`teacher_course_count`	Словарь нагрузки преподавателей (кол - во курсов)
`PerformanceGroup`	`performance_groups`	Группировка студентов по уровню успеваемости
`MultiCourseStudebts`	`MultiCourseStudents`	Список студентов, обучающихся на ≥2 курсах
`StudentCourseMap`	`StudentCourseMap`	Словарь «студент → курсы», показывает распределение студентов по программам

Зачем нужен этот словарь:

- Служит централизованным источником данных для отчёта по центру.
- Позволяет быстро отобразить показатели вовлечённости и успеваемости.
- Используется для уведомления пользователя о результатах работы робота.
- Все ключевые показатели собраны в одном месте, что облегчает дальнейший анализ или экспорт.

Практика показала, как с помощью структурированных данных и простых алгоритмов можно автоматизировать сбор, анализ и визуализацию ключевых показателей образовательного центра.

КОНТРОЛЬНЫЕ ЗАДАНИЯ

Бизнес-контекст

Финансовый стартап предоставляет клиентам онлайн-счета и проводит транзакции.

Компания активно растёт, увеличивается число клиентов и операций.

Руководство хочет:

- Отслеживать активность клиентов;
- Анализировать объёмы транзакций;
- Выявлять VIP-клиентов;
- Контролировать нагрузку по счетам;
- Формировать управленческие отчёты для инвесторов.

Ваша задача — создать систему, которая:

1. Хранит данные о клиентах, счетах и транзакциях;
2. Выполняет аналитику финансовой активности;
3. Формирует итоговый отчёт для руководства.

Общие требования к данным

- Данные создаются студентами самостоятельно с помощью нейросети **DeepSeek**.
- Минимальный объём:
    - ≥30 клиентов;
    - ≥50 счетов;
    - ≥100 транзакций;
    - У клиента может быть несколько счетов;
    - Каждый счёт содержит список транзакций;
    - Часть клиентов активно совершают операции.

Задание 1

Структура данных

```
fintech = {
  "clients": [
    {
      "name": "Иван Иванов",
```

```
        "email": "ivan@mail.ru",
        "accounts": [
          {
            "account_id": "A001",
            "transactions": [
              {"type": "income", "amount": 5000},
              {"type": "expense", "amount": 1200}
            ]
          }
        ]
      }
  ]
}
```

Часть 1. Подготовка данных

- total_clients = len(fintech["clients"])
    - общее количество клиентов → **идёт в отчёт как TotalClients**
- all_accounts = [acc for client in fintech["clients"] for acc in client["accounts"]]
    - общее количество счетов → **идёт в отчёт как TotalAccounts**
- all_transactions = [t for acc in all_accounts for t in acc["transactions"]]
    - общее количество транзакций → **идёт в отчёт как TotalTransactions**
- last_client = fintech["clients"][-1]
    - анализ нового клиента *(не идёт в отчёт напрямую)*

Часть 2. Финансовая аналитика

1. **Баланс каждого счёта → новый ключ в словаре счёта**

```
account["balance"] = sum(
  t["amount"] if t["type"] == "income" else -t["amount"]
  for t in account["transactions"]
)
```

Зачем: видеть текущее состояние средств.

---

2. **Общий баланс клиента → новый ключ в словаре клиента**

```
client["total_balance"] = sum(acc["balance"] for acc in client["accounts"])
```

Зачем: оценить финансовую активность клиента → **идёт в отчёт**

---

3. **Общий оборот клиента (сумма всех операций) → новый ключ**

```
client["total_turnover"] = sum(
    t["amount"]
    for acc in client["accounts"]
    for t in acc["transactions"]
)
```

Зачем: определить уровень активности → **идёт в отчёт**

---

4. **VIP-клиент (максимальный оборот) → отдельный словарь**

```
vip_client = {
    "name": "Иван Иванов",
    "total_turnover": 150000
}
```

Зачем: выявить наиболее ценного клиента → **идёт в отчёт**

---

5. **Клиенты с отрицательным балансом → список**

```
negative_balance_clients = [
    {"name": "Пётр", "total_balance": -2000}
]
```

Зачем: выявить финансовые риски → **идёт в отчёт**

Итоговый отчёт (Задание 1)

```
fintech_basic_report = {
    "TotalClients": total_clients,
    "TotalAccounts": len(all_accounts),
    "TotalTransactions": len(all_transactions),
    "VIPClient": vip_client,
    "NegativeBalanceClients": negative_balance_clients
```

}

Задание 2

Используется структура данных из Задания 1.

Часть 1. Сегментация клиентов

Разделить клиентов по уровню оборота:

```
client_activity_groups = {
   "High": ["Иван"],    # total_turnover ≥ 100000
   "Medium": ["Анна"],  # 30000–99999
   "Low": []            # <30000
}
```

Зачем: стратегическая работа с клиентскими сегментами → **идёт в отчёт**

Часть 2. Анализ типов транзакций

Подсчитать суммарный доход и расход системы:

```
transaction_summary = {
   "TotalIncome": 500000,
   "TotalExpense": 420000
}
```

Зачем: понять финансовый поток компании → **идёт в отчёт**

Часть 3. Карта счетов

Создать карту «клиент → счета»:

```
client_account_map = {
   "Иван Иванов": ["A001", "A002"],
   "Анна Смирнова": ["A010"]
}
```

Зачем: видеть распределение счетов → **идёт в отчёт**

Также определить клиентов с ≥2 счетами → **MultiAccountClients** (список)

Итоговый отчёт (Задание 2)

```
fintech_extended_report = {
    "TotalClients": total_clients,
    "TotalAccounts": len(all_accounts),
    "TotalTransactions": len(all_transactions),
    "VIPClient": vip_client,
    "NegativeBalanceClients": negative_balance_clients,
    "ClientActivityGroups": client_activity_groups,
    "TransactionSummary": transaction_summary,
    "ClientAccountMap": client_account_map,
    "MultiAccountClients": multi_account_clients
}
```

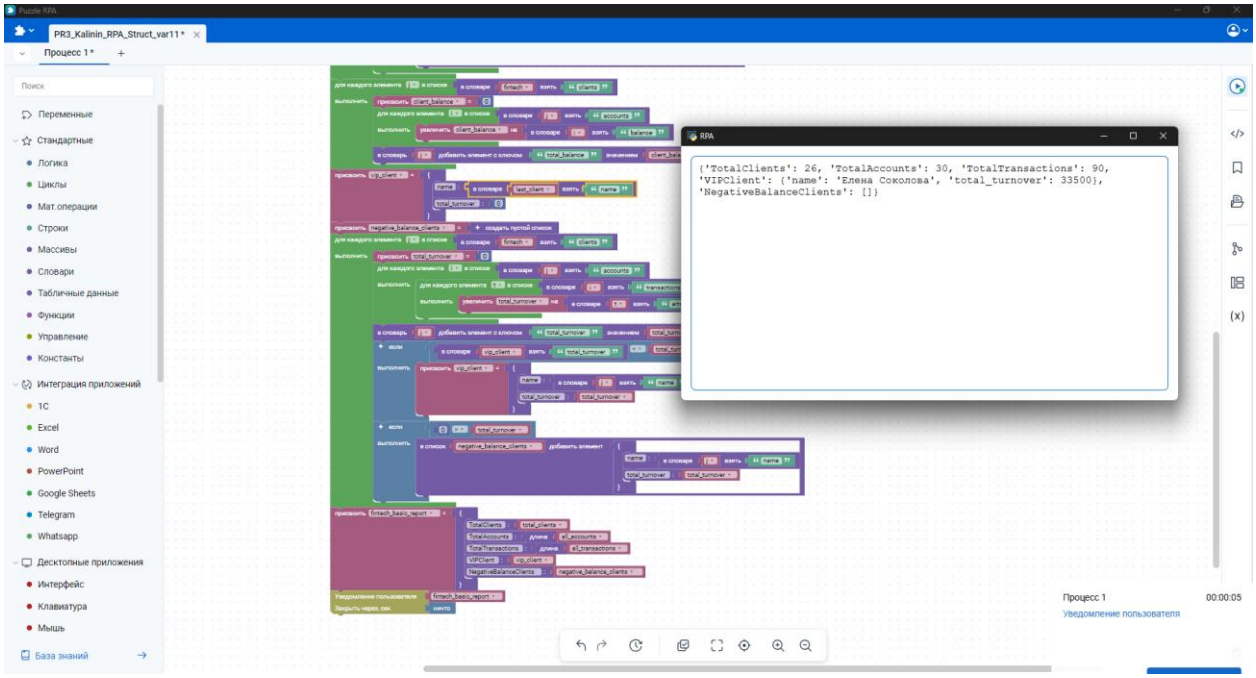
Примечание:

- Ключи отчёта соответствуют переменным.
- Финансовые показатели добавляются как новые ключи в словари счетов и клиентов.
- Сводная аналитика формируется в отдельных словарях.
- Промежуточные переменные используются для расчётов и могут не входить в итоговый отчёт.

Контрольное задание 1

Условие задания варианта: Необходимо разработать аналитическую систему финансового стартапа, которая хранит данные о клиентах, счетах и транзакциях, анализирует финансовую активность, выявляет VIP-клиентов и клиентов с отрицательным балансом, а затем формирует итоговый управленческий отчёт для руководства и инвесторов.

Успешное выполнение кода процесса

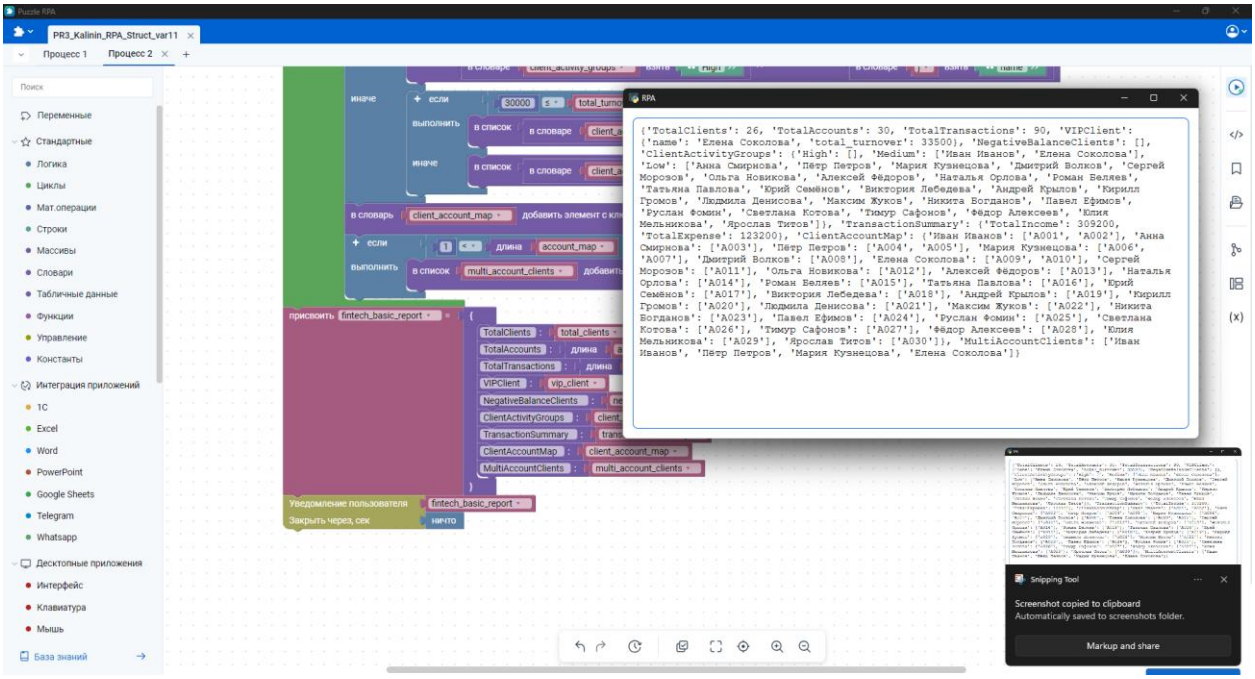


Алгоритм решения: сформировать структуру данных с клиентами, их счетами и транзакциями, затем посчитать общее число клиентов, счетов и операций. После этого для каждого счёта вычислить баланс, для каждого клиента — общий баланс и общий оборот по всем транзакциям. На основе рассчитанных показателей определить VIP-клиента с максимальным оборотом, а также выделить клиентов с отрицательным балансом как зону финансового риска. Результат — итоговый аналитический отчёт для руководства, который показывает масштаб клиентской базы, активность по операциям и проблемные счета.

Контрольное задание 2

Условие задания варианта: Необходимо на основе данных о клиентах, счетах и транзакциях выполнить сегментацию клиентов по обороту, проанализировать общий объём доходов и расходов, построить карту счетов по каждому клиенту и определить клиентов с несколькими счетами, после чего включить все результаты в итоговый отчёт для руководства.

Успешное выполнение кода процесса



Алгоритм решения: использовать подготовленные данные из Задания 1, затем для каждого клиента рассчитать общий оборот и на его основе распределить клиентов по уровням активности High, Medium и Low. После этого подсчитать суммарные доходы и расходы по всем транзакциям, чтобы оценить общий финансовый поток системы. Далее построить карту соответствия «клиент → счета» и отдельно выделить клиентов, у которых открыто два и более счетов. Результат — расширенный аналитический отчёт, который показывает клиентские сегменты, структуру денежных потоков и распределение счетов между клиентами.

Ссылка на репозиторий: https://github.com/NKTKLN/mirea-puzzle-rpa/tree/main/practice_3

Ссылка на чат DeepSeek: https://chat.deepseek.com/share/pt3salbh2wrjpmbw1f